



ClansCore

Weiterentwicklung Discord-Bot für Vereine

Studien- und Bachelorarbeit im
Herbst-Semester 2025

Ausdruck:

24.10.2025

Team / Autoren:

Vanessa Alves und Joel Thoma

Referent:

Prof. Dr. Frieder Loch

Koreferent:

Julia Czerniak-Wilmes

Gegenleser:

Severin Dellsperger

Inhaltsverzeichnis

I	Produkt Dokumentation	1
1	Einleitung	1
1.1	Problemstellung	1
1.2	Vorarbeiten	1
1.3	Ziel der Arbeit und Abgrenzung	2
1.4	Rahmenbedingungen	2
1.5	Stand der Technik	2
2	Anforderungen	4
2.1	Vorgehen	4
2.2	Priorisierung der Anforderungen	4
2.3	Legende zum Erfüllungsgrad der Anforderungen	5
2.4	User Stories	5
2.5	Functional Requirements	5
2.6	Use Cases	8
2.7	Non-Functional Requirements	12
3	Erweitertes Gamification-Konzept	15
3.1	Self-Determination Theory	15
3.2	Interviews zur Motivation von Vereinsmitgliedern	15
3.2.1	SDT-basierte Analyse der Interviewergebnisse	16
3.2.2	Querschnittliche Implikationen	17
3.3	Octalysis Framework	17
3.4	Erweiterungen, Chancen und Regeln	19
3.4.1	Designprinzipien	19
3.4.2	Geplante Erweiterungen und Regeln	20
3.4.3	Technische Implikationen und Roadmap	21
3.4.4	Zukünftige Erfolgsmessung	21
4	Erweitertes Sicherheitskonzept	22
4.1	Ziel und Geltungsbereich	22
4.2	Sicherheitsprinzipien	22
4.3	Authentifizierung und Autorisierung	23
4.3.1	Dienstkonto und automatisierte Zugriffe	23
4.3.2	Absicherung des Datenbankzugriffs	23
4.3.2.1	Bewertung und Entscheid.	23
4.3.2.2	Übertragbarkeit auf andere Vereine	24
4.3.3	Dashboard-Login	24
4.4	Daten- und Schnittstellensicherheit	24
4.4.1	Eingabevalidierung	24
4.4.2	Synchronisierung und Konfliktlösung	24
4.5	Datenschutz und DSGVO / DSGVO-Umsetzung	25
4.6	Server- und Deployment-Sicherheit	25
4.7	Manipulations- und Integritätsschutz	25
4.8	Risikoanalyse und Massnahmen	25
4.9	Monitoring und Wartung	26
4.10	Abgrenzung des Sicherheitskonzeptes	26
5	Systemanalyse, Architektur und Wireframes	27
5.1	Domain Analyse	27
5.2	C4-Modell	28
5.2.1	System Context Diagramm	28
5.2.2	Container Diagramm	29
5.2.3	Component Diagramm	30
5.3	Wireframes	30
5.3.1	Nicht-Mitglied	30
5.3.2	Gamification-Verwaltung	32

6	Qualitätssicherung	34
6.1	Testkonzept	34
6.1.1	Vorgehensweise	34
6.1.2	Testziele	34
6.1.3	Testumgebung	34
6.1.4	Unit- und Integrationstests	34
6.1.5	Usability-Tests	34
6.2	Ablaufplan	35
6.3	Code-Qualität	36
6.4	Definition of Done	36
7	Implementation	37
8	Fazit	38

II Projekt Dokumentation 39

9	Projekt Plan	39
9.1	Zusammenarbeit	39
9.2	Meetings	39
9.3	Minimum Viable Product (MVP)	39
9.4	Long-Term Plan	40
9.4.1	Inception (Initiierung)	41
9.4.2	Elaboration (Ausarbeitung)	41
9.4.3	Construction (Konstruktion)	41
9.4.4	Transition (Übergang)	41
9.4.5	Presentation (Präsentation)	41
9.4.6	Meilensteine	41
9.5	Risikomanagement	42
9.5.1	Ausweichplan	45
10	Arbeitszeitznachweis	46
11	Tooling und Technologie-Entscheidungen	47
11.1	Jira	47
11.2	Teams	47
11.3	GitHub	47
11.3.1	GitHub Guidelines	48
11.4	Dokumentation	48
11.4.1	Dokumentation Guidelines	48
11.4.2	Dokumentation Deployment	48
11.5	Code und Entwicklung	49
11.5.1	Code Guidelines	49
11.5.2	Setup	49
11.5.3	Hosting	49
11.5.4	Code Deployment	49
11.6	Datenbank	50
11.7	Programmiersprachen und Frameworks	50
11.8	UI-Bibliothek	51
11.9	KI-Tools	51
12	Sitzungsprotokolle	52
13	Persönliche Erfahrungsberichte	56
13.1	Bericht Joel Thoma	56
13.2	Bericht Vanessa Alves	56

III Glossar und Abkürzungsverzeichnis 57

IV Literaturverzeichnis 59

V	Abbildungsverzeichnis	61
VI	Tabellenverzeichnis	62
VII	Code-Listings	65
VIII	Anhang	66
14	Interviews zur Vereins-Motivation	66
15	Technische Ergänzungen zum Sicherheitskonzept	66
15.1	Vergleich der DB-MFA-Varianten	66
15.2	Dashboard-Sicherheit und Passwortmanagement	67
15.3	Nebenläufigkeitskontrolle und Transaktionen	68
15.4	Container- und Deployment-Details	68

Kapitel I

Produkt Dokumentation

1 Einleitung

Digitale Kommunikationsplattformen wie Discord haben sich in den vergangenen Jahren zunehmend als zentrale Werkzeuge für die Organisation und Interaktion innerhalb von Online-Communities und Vereinen etabliert. Insbesondere im Bereich der Freizeit- und Gaming-Vereine bieten sie die Möglichkeit, Kommunikation, Eventplanung und Mitgliederverwaltung zu bündeln. Trotz dieser Vorteile stossen bestehende Plattformfunktionen bei wachsender Mitgliederzahl, zunehmender organisatorischer Komplexität und steigenden Anforderungen an Datensicherheit und Automatisierung an ihre Grenzen.

Vor diesem Hintergrund befasst sich die vorliegende Bachelorarbeit mit der Weiterentwicklung eines im Vorprojekt erstellten Discord-Bots, der vereinsrelevante Prozesse automatisiert und Gamification-Mechanismen zur Steigerung des Engagements nutzt. Ziel der Arbeit ist es, das bestehende System, um ein plattformunabhängiges Admin-Dashboard zu erweitern, welches die sichere, benutzerfreundliche und rollenbasierte Verwaltung von Mitgliederdaten und Gamification-Statistiken ermöglicht. Damit wird ein Beitrag zur Entkopplung der Vereinsverwaltung von der Kommunikationsplattform Discord geleistet und die Grundlage für eine modulare, erweiterbare Systemarchitektur geschaffen.

1.1 Problemstellung

Das im Vorprojekt entwickelte System ermöglicht die Automatisierung einiger zentraler Vereinsprozesse, ist jedoch ausschliesslich innerhalb der Discord-Umgebung funktionsfähig. Dadurch ist die Nutzung stark Discord-zentriert und für Personen ohne Discord-Konto oder mit eingeschränkter technischer Erfahrung nur begrenzt zugänglich.

Die Bearbeitung vereinsrelevanter Daten erfolgt derzeit ausschliesslich über direkten Zugriff auf die MongoDB-Datenbank. Dies erfordert spezifische technische Kenntnisse, birgt Sicherheitsrisiken und verhindert eine kontrollierte, benutzerfreundliche Datenverwaltung. Zudem existiert keine zentrale Oberfläche, die eine visuelle und statistisch fundierte Auswertung des Vereinsengagements ermöglicht.

Darüber hinaus erschwert die aktuelle Architektur die Anbindung zusätzlicher Plattformen oder externer Systeme, etwa zur Erweiterung um Web- oder Mobile-Anwendungen. Diese Einschränkungen verdeutlichen den Bedarf nach einer modularen, skalierbaren und plattformunabhängigen Systemlösung, die sowohl technische als auch organisatorische Herausforderungen adressiert.

1.2 Vorarbeiten

Das Vorprojekt „Discord-Bot für Vereine“ (Alves & Schnider, 2025) bildete die Grundlage für die vorliegende Arbeit. Es verfolgte das Ziel, zentrale Vereinsprozesse wie Mitgliederverwaltung, Eventorganisation und Gamification in einem Discord-Bot zu integrieren. Der entwickelte Prototyp implementierte ein Rollen- und Berechtigungssystem, eine Anbindung an die Google Calendar API sowie ein auf wissenschaftlichen Erkenntnissen basierendes Gamification-Konzept.

Während die Machbarkeit und der Nutzen eines solchen Systems bestätigt werden konnten, blieb dessen Erweiterbarkeit auf andere Plattformen sowie die administrative Steuerung offen. Das Admin-Dashboard wurde im Vorprojekt lediglich als potenzielle Weiterentwicklung beschrieben, jedoch nicht realisiert. Ebenso fehlten Mechanismen zur Messung und Visualisierung des Engagements sowie eine Entkopplung der Discord-spezifischen Logik von der Datenbankstruktur.

Die vorliegende Arbeit schliesst an diese Ergebnisse an und adressiert die genannten Defizite durch eine Neugestaltung der Architektur, eine verbesserte Datenkonsistenz zwischen Discord, Dashboard und Datenbank sowie durch die Integration einer administrativen Benutzeroberfläche. Gleichzeitig werden Erkenntnisse aus Interviews mit Präsidenten anderer Vereine in die konzeptionelle Weiterentwicklung des Gamification-Systems einbezogen, um dessen Übertragbarkeit und Nachhaltigkeit zu erhöhen.

1.3 Ziel der Arbeit und Abgrenzung

Ziel dieser Bachelorarbeit ist die Konzeption und Entwicklung einer plattformunabhängigen, administrativen Erweiterung des bestehenden Discord-Bots, die eine effiziente und sichere Verwaltung vereinsrelevanter Daten ermöglicht. Hierbei liegt der Fokus auf der funktionalen Umsetzung eines Admin-Dashboards sowie auf der architektonischen Modularisierung des Systems, um künftige Integrationen weiterer Plattformen leichter zu ermöglichen. Die neue Softwarelösung wird mit dem Namen ClansCore adressiert.

Im Einzelnen verfolgt die Arbeit folgende Teilziele:

- Entwurf und Implementierung eines funktionsfähigen Admin-Dashboards in ClansCore, zur Verwaltung von Mitgliedern, Rollen, Events und Gamification-Daten
- Refaktorisierung der ClansCore-Backend-Architektur zur Entkopplung von Discord-spezifischer Logik und zur Vorbereitung auf eine plattformübergreifende Erweiterung
- Entwicklung einer sicheren Schnittstellenkommunikation zwischen Bot, Dashboard und Datenbank zur Gewährleistung konsistenter Datenzustände innerhalb des gesamten ClansCore-Systems
- konzeptionelle und technische Erweiterung des Gamification-Systems, insbesondere zur Messung des Mitgliederengagements
- Evaluation von ClansCore durch Usability-Tests mit Vereinsmitgliedern

Diese Arbeit unterscheidet sich vom Vorprojekt durch ihren erweiterten technischen und analytischen Anspruch. Während das Vorprojekt primär die Funktionalität innerhalb von Discord validierte, liegt der Schwerpunkt der Bachelorarbeit auf der systematischen Generalisierung und Professionalisierung der Lösung als Gesamt-System namens ClansCore. Damit wird die Grundlage geschaffen, um das System langfristig als universelles Vereinsmanagement-Tool einsetzen zu können.

1.4 Rahmenbedingungen

Die Arbeit wurde im Rahmen der Bachelor- und Studienarbeit des Studiengangs Informatik an der Ostschweizer Fachhochschule (OST) durchgeführt.

- **Arbeitstyp:** Bachelorarbeit (360 Stunden) + Studienarbeit (240 Stunden)
- **Gesamtarbeitsaufwand:** 600 Stunden
- **ECTS-Punkte:** 12 ECTS (Bachelorarbeit) und 8 ECTS (Studienarbeit)

Der Schwerpunkt liegt auf der Konzeption, Entwicklung und Evaluation eines funktionalen Software-Systems mit wissenschaftlich begründetem Gamification-Ansatz, welches den Namen ClansCore trägt. Die Arbeit vereint methodische Ansätze aus den Bereichen Software-Engineering, Usability und Motivationsforschung und ist als entwicklungsorientierte empirische Arbeit mit prototypischem Charakter einzuordnen.

1.5 Stand der Technik

Der Stand der Technik im Bereich digitaler Vereinsorganisation wurde im Vorprojekt umfassend untersucht und bildet die Grundlage der vorliegenden Arbeit. Die damalige Marktanalyse zeigte, dass bestehende Systeme entweder auf die Interaktion und Gamification innerhalb von Discord oder auf die administrative Vereinsverwaltung über klassische Softwarelösungen fokussieren, jedoch keine integrative Verbindung beider Ansätze bieten.

Seit Abschluss des Vorprojekts im Juli 2025 sind zwar Weiterentwicklungen einzelner Systeme erkennbar, eine Lösung, welche Gamification, Datenverwaltung und plattformübergreifende Nutzung in einem kohärenten Konzept vereint, wurde bisher jedoch nicht öffentlich dokumentiert.

Im Bereich der Discord-Bots zählt MEE6 zu den meistgenutzten Anwendungen und bietet über das Plugin „Statistics Channels“ grundlegende Auswertungen zur Serveraktivität sowie ein Levelsystem zur Förderung der Nutzerinteraktion (MEE6-Contributors, 2025). Auch der Bot Arcane integriert vergleichbare Gamification-Funktionen in Form von Belohnungssystemen (Arcane-Contributors, 2025). Beide Systeme bleiben jedoch auf die Discord-Plattform beschränkt und verfügen über keine dokumentierten Schnittstellen zu externen Administrations- oder Datenverwaltungssystemen.

Demgegenüber adressieren klassische Vereinsmanagement-Lösungen wie ClubDesk oder WildApricot die organisatorischen Anforderungen von Vereinen, insbesondere in den Bereichen Mitgliederverwaltung, Eventplanung und Buchhaltung (ClubDesk-Contributors, 2025; WildApricot-Contributors, 2025). Motivierende oder interaktive Komponenten, wie sie in Gamification-Konzepten beschrieben werden, sind in diesen Systemen kein grosser Bestandteil. Diskussionen über eine

mögliche Erweiterung durch Belohnungsmechanismen existieren lediglich in Anwenderforen, was die bislang geringe praktische Umsetzung solcher Konzepte verdeutlicht ([WildApricot-Community, 2015](#)) .

Die vorliegende Arbeit positioniert sich zwischen diesen beiden Ansätzen, indem sie den organisatorischen Verwaltungsfokus klassischer Systeme mit der motivierenden Wirkung von Gamification-Elementen in [ClansCore](#) kombiniert. Durch die laufende Einführung des vorherigen Systems (Discord-Bot) beim Verein [EGSwiss](#) über das Teammitglied Vanessa Alves, während der Sommermonate 2025, konnten Rückmeldungen des Vorstandes gesammelt und praxisrelevante Anforderungen sowie Verbesserungsvorschläge identifiziert werden. Diese sind in der bestehenden Softwarelösung bislang unberücksichtigt geblieben sind. Damit leistet die Arbeit einen Beitrag zur Integration von Motivation, Datenmanagement und Vereinsorganisation in einem einheitlichen, plattformunabhängigen Anwendungskonzept ClansCore.

2 Anforderungen

Dieses Kapitel stellt die Anforderungen an ClansCore für den Verein systematisch dar. Die Anforderungen basieren auf klar definierten User Stories, welche die Bedürfnisse der Nutzer abbilden. Aus diesen User Stories werden die Functional Requirements abgeleitet, welche die konkreten Funktionen des Bots und des Dashboards spezifizieren. Darauf aufbauend definieren die Use Cases, wie diese Anforderungen in der geplanten Umsetzung realisiert werden. Neben den Functional Requirements werden auch die Non-Functional Requirements berücksichtigt, um Qualitätsmerkmale wie Sicherheit, Skalierbarkeit und Benutzerfreundlichkeit sicherzustellen. Zudem erfolgt eine Priorisierung der Anforderungen, damit die wichtigsten Funktionen frühzeitig implementiert werden. Im Gegensatz zum Vorprojekt liegt der Schwerpunkt auf der Erweiterung des bestehenden Discord-Bots zu einem modularen und sicheren Verwaltungssystem mit Web-Dashboard (siehe Abschnitt 1.3).

2.1 Vorgehen

Die Anforderungen wurden aus der Aufgabenstellung, dem Vorprojekt, den Praxiserfahrungen von Vanessa Alves in ihrer Funktion als Präsidentin des Vereins EGSwiss sowie einer ergänzenden Analyse der Domäne abgeleitet. Der Erhebungsprozess erfolgte in mehreren Schritten:

1. User Stories erfassen die Bedürfnisse und Erwartungen der verschiedenen Rollen.
2. Functional Requirements (FR) leiten daraus die technologie-neutralen Systemfunktionen ab.
3. Use Cases (UC) konkretisieren die Interaktionen zwischen Nutzern und System und validieren die FR.
4. Non-Functional Requirements (NFR) definieren überprüfbare Qualitäts- und Leistungsanforderungen.

Im Gegensatz zum Vorprojekt liegt der Schwerpunkt dieser Arbeit auf der Erweiterung des bestehenden Discord-Bots zu einem modularen und sicheren Verwaltungssystem mit Web-Dashboard (siehe Abschnitt 1.3). Zwischen User Stories, Functional Requirements und Use Cases besteht kein 1:1-Mapping. Die Beziehungen sind häufig viele-zu-viele.

Die Umsetzung und Evaluierung konzentriert sich auf die Nutzung des Admin-Dashboards auf Desktop-Systemen. Eine Optimierung für mobile Endgeräte und Cross-Device-Design liegt ausserhalb des Projektumfangs.

2.2 Priorisierung der Anforderungen

Damit der Entwicklungsprozess auf die wichtigsten Features fokussiert bleibt, erhalten gewisse Anforderungen eine höhere Priorität als andere. Dadurch kann zügig ein Prototyp entwickelt, getestet und auf Basis von Nutzerfeedback iterativ verbessert werden. Die verpflichtenden Use Cases und Non-Functional Requirements bilden das MVP (Minimum Viable Product) und stellen die funktionale Basis des Projekts dar.

Die nachfolgende Auflistung beschreibt die Prioritäts-Kategorien und deren Bedeutung:

Prioritäts-Kategorie	Beschreibung
Hoch (<u>MVP</u>) - Essenziell für die erste Version	Diese Anforderungen sind direkt mit den Kernanforderungen des MVP verknüpft und müssen für eine funktionale Erstversion vorhanden sein.
Mittel - Wichtige Funktionen nach dem MVP	Diese Anforderungen ergänzen das System sinnvoll, sind jedoch nicht kritisch für die erste Version.
Niedrig - Erweiterungen für spätere Versionen	Diese Anforderungen verbessern die Benutzerfreundlichkeit und Verwaltung, sind aber nicht erforderlich für den ersten MVP-Release.

Tab. 1: Beschreibung der Prioritäts-Kategorien

2.3 Legende zum Erfüllungsgrad der Anforderungen

Die Resultate der Anforderungen werden in den Tabellen dokumentiert. Die farbliche Bewertung des Erfüllungsgrads erfolgt erst im Fazit, am Ende des Projekts, nicht während der Spezifikation.

Farbe	Status (wird am Projektende gesetzt)
Grün	Ziel der Anforderung wurde erreicht.
Gelb	Ziel der Anforderung wurde zum Teil oder über einen anderen Weg erreicht.
Rot	Ziel der Anforderung wurde nicht erreicht.

Tab. 2: Beschreibung der Resultats-Kategorien

2.4 User Stories

Die nachfolgenden User Stories beschreiben die Erwartungen und Bedürfnisse der Nutzer an das aus dem Vorprojekt erweiterte System ClansCore. Sie werden aus der Sicht der Endanwender formuliert und bilden die Grundlage für die Ableitung der Functional Requirements. Die Analyse berücksichtigt die Rollen Vorstand, Mitglied, Nicht-Mitglied sowie die Systemperspektive Verein.

Als Verein...

- möchte ich ClansCore plattformunabhängig und modular betreiben, um flexibel auf neue Plattformen reagieren zu können.
- möchte ich eine zentrale Übersicht über Rollen, Events, Aufgaben und Aktivitäten haben, um operative und strategische Entscheide zu treffen.
- möchte ich Verlässlichkeit und Vertrauen durch Datenintegrität, Sicherheit und regelmässige Sicherungen / Überwachung sicherstellen.

Als Vorstandsmitglied...

- möchte ich Mitglieder, Rollen und Events im Admin-Dashboard verwalten, inklusive Rechte und Synchronisation mit Discord.
- möchte ich Aufgaben und Gamification erfassen und steuern sowie Analysen zu Aktivität und Engagement erhalten.
- möchte ich Zahlungserinnerungen und Zahlungsbestätigungen automatisiert erhalten, um administrativen Aufwand zu reduzieren.
- möchte ich Nachvollziehbarkeit durch Änderungs- und Transaktionsprotokolle sowie Datenintegritätsprüfungen.
- möchte ich sicheren Administrationszugriff sowie Monitoring und Alarme bei Konflikten und Sicherheitsereignissen.

Als Mitglied...

- möchte ich Aufgaben, Punkte und Ranglisten plattformunabhängig einsehen, jedoch konsistent zu Discord.
- möchte ich transparente Punkteberechnung verstehen und mich für Aufgaben anmelden.
- möchte ich meine Datenschutzrechte (Auskunft, Löschung, Einwilligung) selbst ausüben.

Als Nicht-Mitglied...

- möchte ich mich ausserhalb von Discord bewerben können.

2.5 Functional Requirements

Die Functional Requirements leiten sich aus den User Stories ab und spezifizieren die erwarteten Funktionen von ClansCore. Sie beschreiben, welche neuen oder erweiterten Features im Rahmen dieser Arbeit implementiert werden müssen, um die in der Einleitung definierten Ziele zu erreichen (siehe Abschnitt 1.3).

Die Kernfunktionen umfassen Gamification, Vereins- und Eventverwaltung. Basis-Funktionen (z. B. Passwortänderung oder -zurücksetzung) sichern die erwarteten Standardprozesse ab.

Im Gegensatz zum Vorprojekt liegt der Schwerpunkt auf Plattformunabhängigkeit, Sicherheit, Datenintegrität und Erweiterbarkeit durch ein webbasiertes Dashboard.

ID	FR1
Name	Login und Authentifizierung
Beschreibung	• Das Admin-Dashboard stellt ein sicheres Loginsystem mit Benutzername und Passwort bereit (Passwort-Policy, Sperrmechanismen/Rate-Limiting, sichere Session-Verwaltung). • Das System muss optional eine <u>Multi-Faktor-Authentifizierung</u> (MFA) unterstützen. • Authentifizierungs- und Autorisierungsvorgänge werden nachvollziehbar protokolliert. • Transport der Anmeldedaten erfolgt über verschlüsselte Verbindungen.

Tab. 3: Beschreibung Functional Requirement 1

ID	FR2
Name	Online-Mitgliedsbewerbung
Beschreibung	• Das Dashboard muss ein Bewerbungsformular für Nicht-Mitglieder bereitstellen. • Bewerbungen müssen automatisch in der Datenbank gespeichert und dem Vorstand im Dashboard angezeigt werden.

Tab. 4: Beschreibung Functional Requirement 2

ID	FR3
Name	Passwortmanagement
Beschreibung	• Benutzer des Dashboards müssen ihr Passwort selbstständig ändern und zurücksetzen können. • Passwörter müssen verschlüsselt gespeichert und sicher übermittelt werden.

Tab. 5: Beschreibung Functional Requirement 3

ID	FR4
Name	Verwaltung von Mitgliedern, Rollen und Events
Beschreibung	• Mitgliederprofile und Rollen anzeigen / bearbeiten. • Events erstellen, bearbeiten und löschen. • Erinnerungen / Benachrichtigungen für Events generieren. • Änderungen werden mit der Discord-Datenhaltung konsistent synchronisiert.

Tab. 6: Beschreibung Functional Requirement 4

ID	FR5
Name	Steuerung und Rechteverwaltung
Beschreibung	• Das Dashboard muss erlauben, die Zugriffsrechte für Bot-Funktionen und Dashboard-Bereiche zu verwalten. • Es muss konfigurierbar sein, welche Rollen welche Aktionen ausführen dürfen. • Änderungen an Berechtigungen müssen automatisch protokolliert werden.

Tab. 7: Beschreibung Functional Requirement 5

ID	FR6
Name	Gamification-Parameterverwaltung
Beschreibung	• Das Dashboard muss die Erfassung, Anpassung und Gewichtung von Gamification-Parametern ermöglichen. • Bei der Erstellung von Aufgaben muss das System automatisch eine empfohlene Punktezahl vorschlagen.

Tab. 8: Beschreibung Functional Requirement 6

ID	FR7
Name	Gamification-Datenanalyse
Beschreibung	• Detaillierte Statistiken zu Aufgaben, Belohnungen und Aktivitäten. • Near-Real-Time-Auswertung und Visualisierung von Mitgliederaktivitäten (End-to-End-Latenz ≤ 5 Sekunden für Ereignisse unter Normalbetrieb und keine Anforderungen eines Hard-Echtzeitsystems).

Tab. 9: Beschreibung Functional Requirement 7

ID	FR8
Name	Gamification-Ansicht für Mitglieder
Beschreibung	• Mitglieder sehen Punkte, Aufgaben, Belohnungen (Rewards), Ranglisten (Leaderboard), Punkte-Historie sowie die Regeln der Punktevergabe. • Alle Werte stammen aus der zentralen Datenquelle (Konsistenz zu Discord).

Tab. 10: Beschreibung Functional Requirement 8

ID	FR9
Name	Aufgabenverwaltung und Anmeldung
Beschreibung	• Das Dashboard muss die Erstellung und Verwaltung von Aufgaben ermöglichen. • Mitglieder müssen sich über das Dashboard für Aufgaben anmelden können. • Der Fortschritt und Abschlussstatus muss vom Vorstand überprüft und freigegeben werden können.

Tab. 11: Beschreibung Functional Requirement 9

ID	FR10
Name	Zahlungserinnerung
Beschreibung	• Das System muss periodisch prüfen, ob ein Mitglied seinen Mitgliederbeitrag bezahlt hat. • Bei ausstehenden Zahlungen muss automatisch eine Erinnerung über Discord oder das Dashboard versendet werden. • Die Erinnerung darf erst nach Ablauf des definierten Vereinsjahres erfolgen.

Tab. 12: Beschreibung Functional Requirement 10

ID	FR11
Name	Zahlungsbestätigung
Beschreibung	• Nach der Erfassung einer eingegangenen Zahlung muss das System dem Vorstand automatisch eine Bestätigung übermitteln. • Die Zahlungsinformationen müssen revisionssicher gespeichert werden.

Tab. 13: Beschreibung Functional Requirement 11

ID	FR12
Name	Plattformunabhängigkeit und Erweiterbarkeit
Beschreibung	• Architektur erlaubt Integration weiterer Plattformen neben Discord. • Gemeinsame Funktionen (Mitglieder, Aufgaben, Rollen, Gamification) sind über eine zentrale Logik- / Datenschicht verfügbar. • Kommunikation über klar definierte Schnittstellen (z. B. REST, WebSocket, Event-Queue). • Neue Plattformen sollen ohne tiefgreifende Änderungen am Kernsystem hinzugefügt werden können (Implementierung neuer Interfaces, ohne Anpassung bestehender Kern-Pakete).

Tab. 14: Beschreibung Functional Requirement 12

2.6 Use Cases

Die Use Cases beschreiben detailliert die Interaktionen zwischen Nutzern und ClansCore. Sie zeigen, wie die **Functional Requirements** umgesetzt werden. Darüber hinaus konkretisieren sie Anforderungen an Sicherheit, Skalierbarkeit und Benutzerfreundlichkeit.

ID	UC1	
Name	Login und Authentifizierung (<u>FR1</u>)	
Akteur	Mitglied, Vorstand	
Beschreibung	Der Zugriff auf das Admin-Dashboard erfordert eine Authentifizierung (optional wird <u>MFA</u> unterstützt).	
Voraussetzung	Ein gültiger Dashboard-Account besteht.	
Standard-Verlauf	1. Der Benutzer gibt Benutzername und Passwort ein. 2. ClansCore prüft die Zugangsdaten. 3. (Optional) Das System fordert den zweiten Faktor an und validiert ihn. 4. Bei Erfolg wird der Benutzer zur Startseite weitergeleitet.	
Alternativ-Verlauf	• Ungültige Zugangsdaten führen zur Fehlermeldung. • Benutzer hat das Passwort vergessen und kann es zurücksetzen (<u>UC3</u>).	
Nachbedingung	Der Benutzer ist eingeloggt und der Zugriff wird protokolliert.	
Priorität	Hoch	
Resultat		

Tab. 15: Beschreibung Fully dressed Use Case 1

ID	UC2	
Name	Online-Mitgliedsbewerbung (<u>FR2</u>)	
Akteur	Nicht-Mitglied, Vorstand	
Beschreibung	Nicht-Mitglieder bewerben sich ausserhalb von Discord. Der Vorstand wird kanalneutral informiert (Dashboard-Hinweis / Discord-Nachricht / E-Mail). Die konkrete Ausprägung ist konfigurierbar.	
Voraussetzung	Dashboard ist mit der Vereinsdatenbank verbunden.	
Standard-Verlauf	1. Nicht-Mitglied füllt das Bewerbungsformular aus und stimmt der Datenverarbeitung zu. 2. ClansCore speichert die Bewerbung und informiert den Vorstand im Dashboard. 3. Der Vorstand prüft die Angaben und entscheidet (Annahme / Ablehnung). 4. Bei Annahme wird der Mitgliedsstatus gesetzt und die Synchronisation mit Discord ausgelöst. 5. Bewerbende erhalten eine Status-Benachrichtigung über den Entscheid.	
Alternativ-Verlauf	• Unvollständige / fehlerhafte Angaben führen zu einem Validierungsfehler und Korrekturaufforderung. • Ablehnung führt zu einer Benachrichtigung mit Begründung (sofern vorgesehen).	
Nachbedingung	Es wird im System dokumentiert und Rollen / Synchronisation sind konsistent.	
Priorität	Hoch	
Resultat		

Tab. 16: Beschreibung Fully dressed Use Case 2

ID	UC3
Name	Passwort zurücksetzen (<u>FR3</u>)
Akteur	Mitglied, Vorstand
Beschreibung	Benutzer setzen ein vergessenes Passwort sicher zurück.
Voraussetzung	Ein Dashboard-Account existiert.
Standard-Verlauf	1. Benutzer startet „Passwort vergessen“. 2. Das System stellt einen sicheren Reset-Prozess bereit. 3. Benutzer setzt ein neues Passwort und meldet sich an (<u>UC1</u>).
Alternativ-Verlauf	• Wenn kein Account vorhanden ist, führt das zum Abbruch mit Hinweis. • Wenn der Token abgelaufen / ungültig ist, wird ein neuer Reset-Prozess erforderlich.
Nachbedingung	Das neue Passwort ist gespeichert und das Ereignis protokolliert.
Priorität	Hoch
Resultat	

Tab. 17: Beschreibung Fully dressed Use Case 3

ID	UC4
Name	Passwort ändern (<u>FR3</u>)
Akteur	Mitglied, Vorstand
Beschreibung	Benutzer ändern ihr Passwort selbstständig.
Voraussetzung	Benutzer ist eingeloggt.
Standard-Verlauf	1. Benutzer gibt altes und neues Passwort ein. 2. Das System verifiziert das alte Passwort und speichert das neue.
Alternativ-Verlauf	• Falls das alte Passwort falsch ist, wird eine Fehlermeldung ausgegeben und die Änderung abgebrochen.
Nachbedingung	Das neue Passwort ist aktiv und die Änderung protokolliert.
Priorität	Mittel
Resultat	

Tab. 18: Beschreibung Fully dressed Use Case 4

ID	UC5
Name	Verwaltung von Mitgliedern, Rollen und Rechten (<u>FR4</u> , <u>FR5</u>)
Akteur	Vorstand
Beschreibung	Der Vorstand verwaltet Mitglieder, Rollen, Events und Berechtigungen zentral im Dashboard.
Voraussetzung	Dashboard ist mit der Vereinsdatenbank verbunden und das Vorstandsmitglied ist eingeloggt.
Standard-Verlauf	1. Vorstand ruft den Verwaltungsbereich auf. 2. Anzeigen / Bearbeiten von Mitgliedern, Rollen und Events. 3. Konfiguration von Zugriffsrechten und erlaubten Aktionen. 4. Änderungen werden gespeichert und synchronisiert.
Alternativ-Verlauf	• Ein Validierungsfehler führt zur Fehlermeldung und die Änderung wird nicht übernommen.
Nachbedingung	Änderungen sind konsistent persistiert und protokolliert.
Priorität	Hoch
Resultat	

Tab. 19: Beschreibung Fully dressed Use Case 5

ID	UC6
Name	Gamification-Parameter verwalten (FR6)
Akteur	Vorstand
Beschreibung	Der Vorstand pflegt die Gewichtungen / Parameter, welche für Punktevorschläge verwendet werden.
Voraussetzung	Vorstandsmitglied ist eingeloggt.
Standard-Verlauf	1. Öffnen der Parameterverwaltung. 2. Anpassen der Parameter und Speichern.
Alternativ-Verlauf	Unvollständige / fehlerhafte Eingaben ergeben einen Validierungsfehler.
Nachbedingung	Parameter sind aktuell und künftige Punktevorschläge berücksichtigen die Anpassungen.
Priorität	Hoch
Resultat	

Tab. 20: Beschreibung Fully dressed Use Case 6

ID	UC7
Name	Aufgabenverwaltung mit Punktevorschlag (FR6 , FR9)
Akteur	Vorstand
Beschreibung	Der Vorstand erstellt und veröffentlicht Aufgaben. ClansCore schlägt basierend auf Parametern eine Punktezahl vor.
Voraussetzung	Dashboard ist verbunden und das Vorstandsmitglied ist eingeloggt. Die Gamification-Parameter sind gepflegt.
Standard-Verlauf	1. Vorstand erfasst Aufgabe (Metadaten, Kriterien). 2. System zeigt einen berechneten Punktevorschlag. 3. Vorstand bestätigt / ändert und veröffentlicht die Aufgabe.
Alternativ-Verlauf	• Wenn keine Parameter vorhanden sind, gibt es keinen Vorschlag. Die Aufgabe kann dennoch mit manueller Punktezahl angelegt werden. • Ein Validierungsfehler führt zur Fehlermeldung.
Nachbedingung	Aufgabe ist veröffentlicht und im System gespeichert.
Priorität	Hoch
Resultat	

Tab. 21: Beschreibung Fully dressed Use Case 7

ID	UC8
Name	Ansicht Aufgaben, Ranglisten und eigene Punkte (FR7 , FR8)
Akteur	Mitglied, Vorstand
Beschreibung	Mitglieder sehen Aufgaben, Leaderboards, eigene Punkte, Rewards, Historie und die Regeln der Punktevergabe. Der Vorstand erhält zusätzliche Analyseansichten.
Voraussetzung	Dashboard ist verbunden und der Benutzer ist eingeloggt.
Standard-Verlauf	1. Mitglied ruft die Gamification-Ansicht auf und sieht Aufgaben, Ranglisten sowie den eigenen Punktestand. 2. Vorstand ruft Analyse-Dashboard auf (Aggregationen / Trends).
Alternativ-Verlauf	• Wenn keine Aufgaben verfügbar sind, werden entsprechende Hinweise angezeigt.
Nachbedingung	Gamification-Daten werden konsistent und aktuell angezeigt.
Priorität	Hoch
Resultat	

Tab. 22: Beschreibung Fully dressed Use Case 8

ID	UC9
Name	Anmeldung zu Aufgaben (FR9)
Akteur	Mitglied, Vorstand
Beschreibung	Mitglieder melden sich über das Dashboard für Aufgaben an. Die Freigabe erfolgt durch den Vorstand.
Voraussetzung	Mitglied ist eingeloggt und die Aufgabe veröffentlicht.
Standard-Verlauf	1. Mitglied wählt eine Aufgabe und meldet sich an. 2. System speichert die Anmeldung. 3. Vorstand prüft Abschluss und gibt Punkte frei.
Alternativ-Verlauf	• Ein Fehler bei der Anmeldung führt zu einer Fehlermeldung und erneute Eingabe.
Nachbedingung	Anmeldung / Abschlussstatus sind korrekt gespeichert und Punkte entsprechend gutgeschrieben.
Priorität	Hoch
Resultat	

Tab. 23: Beschreibung Fully dressed Use Case 9

ID	UC10
Name	Benachrichtigungen und Informationen zu Zahlungen (FR10 , FR11)
Akteur	Mitglied, Vorstand, System
Beschreibung	Beiträge werden jährlich fällig. ClansCore automatisiert die Erinnerung und die revisionssichere Rückmeldung zu eingegangenen Zahlungen.
Voraussetzung	Ein Mitglied hat einen ausstehenden Mitgliederbeitrag. Zahlungsdaten sind im System hinterlegt.
Standard-Verlauf	1. Das System prüft periodisch Fälligkeiten und identifiziert ausstehende Zahlungen. 2. Das System versendet eine Zahlungserinnerung (Discord oder Dashboard-Benachrichtigung). 3. Die Zahlung wird erfasst (z. B. manuelle Verbuchung oder Import). 4. Das System speichert Datum und Referenz revisionssicher. 5. Der Vorstand erhält automatisch eine Zahlungsbestätigung im Dashboard.
Alternativ-Verlauf	• Zahlung kann nicht eindeutig zugeordnet werden: der Vorstand wird zur Klärung aufgefordert (offene Position im Dashboard). • Bei erneuter Nichtzahlung nach Erinnerung erscheint der Fall in der Liste „Überfällig“. Der Vorstand entscheidet das weitere Vorgehen.
Nachbedingung	Der Zahlungsstatus des Mitglieds ist aktuell und protokolliert.
Priorität	Mittel
Resultat	

Tab. 24: Beschreibung Fully dressed Use Case 10

ID	UC11
Name	Weitere Plattform-Integration (<u>FR12</u>)
Akteur	Nicht-Mitglied, Mitglied, System
Beschreibung	ClansCore stellt prototypisch Funktionen ausserhalb von Discord oder Dashboard bereit (Bewerbung, Datenabfrage) und synchronisiert zentrale Daten.
Voraussetzung	Neue Plattform ist mit der zentralen Datenbank verbunden.
Standard-Verlauf	1. Nicht-Mitglied reicht Bewerbung über die Plattform ein (<u>UC2</u>). 2. System persistiert Daten und informiert den Vorstand. 3. Vorstand entscheidet. Bei Annahme wird Mitgliedsstatus gesetzt und Daten synchronisiert. 4. Mitglied kann auf der neuen Plattform eigene Personendaten einsehen.
Alternativ-Verlauf	• Ablehnung führt zu keiner Aufnahme und die bewerbende Person wird informiert.
Nachbedingung	Zentrale Daten sind konsistent und Plattformunabhängigkeit ist prototypisch nachgewiesen.
Priorität	Niedrig
Resultat	

Tab. 25: Beschreibung Fully dressed Use Case 11

2.7 Non-Functional Requirements

Die Non-Functional Requirements definieren die Qualitätsmerkmale von ClansCore. Sie betreffen Sicherheit, Wartbarkeit, Zuverlässigkeit, Kompatibilität und Benutzerfreundlichkeit. Jede Anforderung ist überprüfbar und wird im Projektverlauf gemessen oder getestet.

ID	NFR1
Beschreibung	Die Codequalität erfüllt die im Projekt definierten Linting- und Formatierungsregeln. In der CI-Pipeline dürfen keine Fehler vom Typ „error“ auftreten.
Anforderungen	Maintainability - Code Quality
Priorität	Hoch
Messung	Automatische Prüfung mit Linter in der CI-Pipeline. Ziel: 0 Fehler im Hauptbranch.
Testen	Bei jedem Commit.
Resultat	

Tab. 26: Beschreibung Non-Functional Requirement 1

ID	NFR2
Beschreibung	Die Testabdeckung der Kernkomponenten (Bot, Dashboard) beträgt mindestens 70%.
Anforderungen	Maintainability - Testability
Priorität	Mittel
Messung	Automatische Berechnung über das Test-Framework (Coverage Report).
Testen	Bei jedem Release und nach grösseren Codeänderungen.
Resultat	

Tab. 27: Beschreibung Non-Functional Requirement 2

ID	NFR3
Beschreibung	Das System verursacht keine unbehandelten Fehler, die zu Absturz oder Datenverlust führen. Bei Ausfällen bleiben persistente Daten konsistent (z. B. Transaktionen / atomare Updates).
Anforderungen	Reliability - Fault Tolerance, Integrity
Priorität	Hoch
Messung	- Crash-Free-Kriterium: In Fehler- / Chaos-Szenarien (z. B. DB nicht erreichbar, Netzwerk-Timeout, paralleles Update) treten keine unbehandelten Exceptions im Log auf. - Integritäts-Stichprobe: 5 zufällige Vorgänge (z. B. Rollenänderung, Aufgabenabschluss, Punk-tevergabe) bleiben trotz Fehlerpfaden konsistent.
Testen	Am Ende der Implementierungsphase.
Resultat	

Tab. 28: Beschreibung Non-Functional Requirement 3

ID	NFR4
Beschreibung	Das Admin-Dashboard verwendet ein einheitliches und benutzerfreundliches Design mit klarer Navigation und unmittelbarem Feedback.
Anforderungen	Usability - Consistency
Priorität	Hoch
Messung	Usability-Test mit mindestens 3 Testpersonen. Erfolgsziel: 80% der Aufgaben ohne Hilfe innerhalb einer definierten Zeit (Median bei ≤ 2 Minuten) lösbar.
Testen	Am Ende der Implementierungsphase.
Resultat	

Tab. 29: Beschreibung Non-Functional Requirement 4

ID	NFR5
Beschreibung	Das Dashboard ist mit aktuellen Versionen von Chrome und Firefox vollständig nutzbar. Layout und Interaktionen sind Desktop-first optimiert. Die mobile Nutzung liegt ausserhalb des Projekt-umfangs.
Anforderungen	Compatibility - Interoperability
Priorität	Hoch
Messung	Manueller Funktionstest auf beiden Browsern. Erfolgsziel: 100 % Funktionsumfang verfügbar.
Testen	Am Ende der Implementierungsphase.
Resultat	

Tab. 30: Beschreibung Non-Functional Requirement 5

ID	NFR6
Beschreibung	Personenbezogene Daten werden vertraulich behandelt und nach dem Prinzip „Privacy by Design“ verarbeitet. Daten werden verschlüsselt gespeichert und übertragen.
Anforderungen	Security - Confidentiality
Priorität	Hoch
Messung	Prüfung der Verschlüsselungskonfiguration (HTTPS, Datenbankverschlüsselung).
Testen	Vor Projektabschluss.
Resultat	

Tab. 31: Beschreibung Non-Functional Requirement 6

ID	NFR7
Beschreibung	Das Gamification-System darf nicht manuell manipuliert werden können. Punkteänderungen müssen nachvollziehbar protokolliert sein.
Anforderungen	Security - Integrity
Priorität	Hoch
Messung	Überprüfung von 5 Stichproben in der Datenbank. Jede Änderung muss eindeutig einem Benutzer oder Prozess zuordenbar sein.
Testen	Am Ende der Implementierungsphase.
Resultat	

Tab. 32: Beschreibung Non-Functional Requirement 7

ID	NFR8
Beschreibung	Nach einem erfolgreichen Commit und bestandenem Testlauf wird automatisch ein neues Build-Image erstellt und ausgeliefert.
Anforderungen	Maintainability - Deployment Automation
Priorität	Mittel
Messung	CI/CD-Pipeline-Auswertung: Build- und Deploy-Prozess müssen ohne Fehler abgeschlossen werden.
Testen	Nach jedem Commit auf dem Main Branch.
Resultat	

Tab. 33: Beschreibung Non-Functional Requirement 8

ID	NFR9
Beschreibung	Alle Änderungen an der Datenbank müssen im Log nachvollziehbar sein (Benutzer, Zeit, Aktion).
Anforderungen	Security - Accountability
Priorität	Mittel
Messung	Manuelle Kontrolle von 10 zufälligen Datenbankeinträgen. Jede Änderung muss einem Benutzer zugeordnet sein.
Testen	Am Ende der Implementierungsphase.
Resultat	

Tab. 34: Beschreibung Non-Functional Requirement 9

ID	NFR10
Beschreibung	Das Dashboard reagiert auf Benutzerinteraktionen innerhalb von 1 Sekunde und liegt damit innerhalb der von der Nielsen Norman Group empfohlenen Reaktionszeitgrenze. <u>(Jakob, 1993)</u>
Anforderungen	Performance - Response Time
Priorität	Mittel
Messung	Automatisierte Lighthouse- oder Playwright-Tests bei den wichtigsten Aktionen (Login, Seitenwechsel).
Testen	Am Ende der Implementierungsphase.
Resultat	

Tab. 35: Beschreibung Non-Functional Requirement 10

3 Erweitertes Gamification-Konzept

Bereits im Vorprojekt wurden erste Erkenntnisse zu Motivation, Hindernissen und Gamification im Vereinskontext erhoben. Diese basierten auf einer Nutzergruppenanalyse mit Nicht-Mitgliedern, Mitgliedern und Vorstand. Die Ergebnisse lieferten eine breite quantitative Datengrundlage, die allgemeine Tendenzen sichtbar machte. Beispielsweise, welche Aktivitäten besonders motivierend wirken oder welche Risiken bei der Einführung von Gamification-Systemen bestehen. (Alves & Schnider, 2025)

Während das Vorprojekt primär die Sicht der Mitglieder sowie die Erfahrungen von Vanessa Alves als Präsidentin von EGSwiss abbildete, verfolgt die vorliegende Arbeit eine andere Perspektive. Sie untersucht qualitativ, wie Vereinsvorstände Motivation aus organisatorischer Sicht verstehen, welche strukturellen Bedingungen sie fördern oder behindern und wie digitale Systeme, wie der Discord-Bot oder das geplante Admin-Dashboard, sie dabei unterstützen können.

Ziel ist es, die im Vorprojekt identifizierten Motivations-Muster kritisch zu hinterfragen und auf ihre langfristige Umsetzbarkeit im System ClansCore zu prüfen. Dafür wurde die Self-Determination Theory (SDT) als theoretische Grundlage gewählt. Sie beschreibt die psychologischen Bedingungen, die Motivation fördern oder hemmen, und diente als Raster zur Entwicklung und Auswertung der Interviews mit Vereinsvorständen.

Die Erkenntnisse aus diesen Interviews werden im Anschluss mithilfe des Octalysis Frameworks nach Yu-kai Chou in konkrete, technisch umsetzbare Gamification-Mechaniken überführt. Das erweiterte Konzept versteht Gamification somit nicht mehr als isoliertes Punktesystem, sondern als integralen Bestandteil eines digitalen Vereinsökosystems, das psychologische, organisatorische und soziale Faktoren verbindet.

3.1 Self-Determination Theory

Die Self-Determination Theory (SDT) nach Deci und Ryan beschreibt Bedingungen, die Motivation entweder unterstützen oder vermindern können. Im Mittelpunkt der Theorie stehen drei zentrale psychologische Grundbedürfnisse:

- **Autonomie:** Das Bedürfnis, selbstbestimmt handeln zu können und Entscheidungen eigenständig zu treffen.
- **Kompetenz:** Das Bedürfnis, sich fähig zu fühlen und Herausforderungen erfolgreich zu bewältigen.
- **Soziale Eingebundenheit:** Das Bedürfnis, sich mit anderen verbunden zu fühlen und Teil einer Gemeinschaft zu sein.

Werden diese Bedürfnisse erfüllt, erhöht sich die Wahrscheinlichkeit für intrinsisch motiviertes Verhalten. Dadurch können Engagement, Lernprozesse und psychisches Wohlbefinden positiv beeinflusst werden. (Weir, 2025)

Auch im Kontext digitaler Technologien lässt sich die SDT anwenden. Sie geht davon aus, dass Nutzer eine Technologie dann langfristig nutzen, wenn die Interaktion mit dem System zur Befriedigung ihrer psychologischen Grundbedürfnisse beiträgt. Das Interface- und Interaktionsdesign wirkt dabei unmittelbar auf das Erleben von Autonomie und Kompetenz. Eine unklare Bedienung oder mangelndes Feedback kann diese Bedürfnisse frustrieren und zur Ablehnung der Technologie führen. Die soziale Eingebundenheit ist besonders relevant, wenn das System Austausch und Anerkennung zwischen Nutzenden fördert. (Peters Dorian, 2018)

Da die SDT sowohl intrinsische als auch extrinsische Motivation berücksichtigt, bietet sie eine geeignete Grundlage, um zu untersuchen, wie Vereinsmitglieder und Vorstände durch digitale Systeme motiviert werden können. Aufbauend auf diesen theoretischen Annahmen wurden qualitative Interviews mit Vereinsvorständen durchgeführt, um die theoretischen Konzepte mit der Praxis abzugleichen.

Diese drei Bedürfnisse dienten als analytische Kategorien in der Auswertung der Interviews. Aussagen der Befragten wurden entlang dieser Dimensionen kodiert und hinsichtlich ihrer Bedeutung für Vereinsmotivation interpretiert.

3.2 Interviews zur Motivation von Vereinsmitgliedern

Die Interviews sollen einen praxisnahen Abgleich zwischen theoretischen Konzepten und der organisatorischen Realität schaffen. Während das Vorprojekt insbesondere die Perspektive der Mitglieder beleuchtete (Alves & Schnider, 2025), liegt der Fokus nun auf der Führungsebene. Es wurden leitfadengestützte Interviews mit vier Präsidenten verschiedener Schweizer Gaming-Vereine durchgeführt. Ziel war es, herauszufinden, welche Faktoren in der Vereinsrealität tatsächlich motivierend wirken, welche Herausforderungen bestehen und wie Gamification-Ansätze sinnvoll in diesen Kontext integriert werden können.

Die Interviews dienen als empirische Ergänzung zur theoretischen Analyse und schaffen einen praxisnahen Abgleich zwischen psychologischen Konzepten und der organisatorischen Realität von Gaming-Vereinen. Während das Vorprojekt besonders die Sicht der Mitglieder untersuchte (Alves & Schnider, 2025), liegt der Fokus in dieser Arbeit auf der Führungsebene.

Es wurden vier halbstrukturierte Interviews mit Präsidenten verschiedener Schweizer Gaming-Vereine durchgeführt. Ziel war es, zu verstehen, welche Faktoren in der Vereinsrealität tatsächlich motivierend wirken, welche Herausforderungen bestehen und wie Gamification-Ansätze sinnvoll in bestehende Vereinsstrukturen integriert werden können.

Die Auswertung erfolgte qualitativ-interpretativ anhand der Self-Determination Theory (SDT). Sie diente als Struktur für die Leitfragen und als Kategoriensystem für die Analyse der Antworten. Wiederkehrende Themen und Muster wurden entlang der Dimensionen Autonomie, Kompetenz und sozialer Eingebundenheit kodiert und im Kontext der digitalen Vereinsarbeit interpretiert. (siehe Anhang [Abschnitt 14](#))

Dadurch wird sichtbar, wie Vorstände Motivation gezielt fördern, wo sie strukturelle Herausforderungen sehen und welche digitalen Mechanismen sie als besonders hilfreich oder hinderlich einschätzen. Die nachfolgenden Punkte geben die zentralen Erkenntnisse der Interviews wieder:

- **Soziale Faktoren dominieren:** Freundschaften, Identifikation mit dem Verein und gemeinsame Erlebnisse sind der stärkste Treiber für langfristige Teilnahme.
- **Kommunikation und Erinnerungen sind Engpässe:** Informationen gehen ansonsten unter und die Teilnehmerzahl steigt mit klaren, mehrstufigen Remindern.
- **Freiwilligkeit und Sinn:** Ein Zwang demotiviert und der Sinnbezug (Beitrag für den Verein) ist entscheidend.
- **Anerkennung sichtbar machen:** Symbolische Wertschätzung (Badges, Rollenfarben, öffentliche Danksagung) wirkt stärker als materielle Belohnung.
- **Fairness durch Saisons / Resets:** Periodische Zurücksetzungen senken Einstiegshürden für Neu-Mitglieder und verhindern Schneeball-Effekte.
- **Transparenz statt Kontrolle:** Aggregierte Kennzahlen werden gewünscht. Dauerhafte personenbezogene Überwachung wird abgelehnt.

3.2.1 SDT-basierte Analyse der Interviewergebnisse

Die Auswertung der vier geführten Interviews mit Präsidenten von Gaming-Vereinen wurden qualitativ kodiert, thematisch verdichtet und interpretativ in Bezug zu den theoretischen Konzepten gesetzt. Eine detaillierte Auswertung ist im Anhang zu finden (siehe Anhang [Abschnitt 14](#)). In diesem Kapitel werden die Ergebnisse zwecks Übersichtlichkeit zusammengefasst.

SDT-Bedürfnis	Interview-Erkenntnis
Autonomie	Freiwilligkeit, Wahlmöglichkeiten und flexible Beteiligungswege wurden als zentral benannt. Digitale Tools sollen unterstützen statt steuern. Transparenz und optionale Pfade (z. B. Aufgaben wählen, persönliche Ziele setzen) fördern Selbstbestimmung. Starre Vorgaben oder Workflows reduzieren sie.
Kompetenz	Motivation steigt, wenn Fortschritt sichtbar ist (Feedback, Stats, Meilensteine) und Leistungen kontextualisiert werden (Beitrag fürs Team / den Verein statt anstatt nur Punkte für den einzelnen). Ranglisten erzeugen eher kurzfristige Aktivität. Nachhaltiger sind wiederum Aufgaben- / Lernpfade mit klaren Ziel-Feedback-Schleifen.
Soziale Eingebundenheit (Relatedness)	Gemeinschaftsgefühl entsteht vor allem durch gemeinsame Aktivitäten (LAN-Parties, Ausflüge) und öffentliche Anerkennung digital sichtbarer Beiträge. Online-Kanäle (Discord) sind notwendig, ersetzen aber Offline-Erlebnisse nicht. Digitale Features sollen Beziehungen sichtbar und anschlussfähig machen (z. B. Team-Achievements, öffentliche Danksagungen).

Tab. 36: Mapping der Interviewergebnisse zu den SDT-Grundbedürfnissen

Über alle vier Interviews hinweg zeigte sich, dass soziale Eingebundenheit den stärksten Einfluss auf Motivation hat, während Autonomie und Kompetenz vor allem durch Transparenz, Feedback und flexible Teilnahmemöglichkeiten gefördert werden.

3.2.2 Querschnittliche Implikationen

Aus der Analyse lassen sich drei zentrale Muster ableiten, die sich durch alle Interviews ziehen:

- 1) **Kommunikation als Motor:** Mehrstufige Erinnerungen mit klaren Call-to-Actions erhöhen die Teilnahmebereitschaft signifikant.
- 2) **Fairnessmechaniken:** Zeitlich begrenzte Seasons oder Resets senken Einstiegshürden und verhindern kumulative Vorteils-effekte („Reiche werden reicher“).
- 3) **Privatsphäre-wahrende Transparenz:** Gewünscht sind aggregierte Kennzahlen zur Steuerung und Motivation, jedoch ohne personenbezogenes Echtzeit-Tracking.

Die Beobachtung deckt sich mit der in allen Interviews geäußerten Präferenz, Kommunikation über Motivation zu stellen. Erst sichtbare, wertschätzende Strukturen schaffen die Basis, auf der spielerische Elemente Wirkung entfalten können.

Diese Muster spiegeln die Ergebnisse des Vorprojekts wider, insbesondere die Bedeutung sozialer Erlebnisse und die Risiken rein wettbewerblicher Logiken ([Alves & Schnider, 2025](#)), und liefern zugleich konkrete Ansatzpunkte für die Weiterentwicklung von ClansCore.

Obwohl die Stichprobe mit vier Interviews begrenzt ist, bieten die Ergebnisse wertvolle qualitative Einsichten in die Perspektive von Vereinsleitungen. Sie bilden damit eine empirisch fundierte Grundlage für Hypothesen und konzeptionelle Erweiterungen, die in zukünftigen quantitativen Untersuchungen überprüft werden können.

3.3 Octalysis Framework

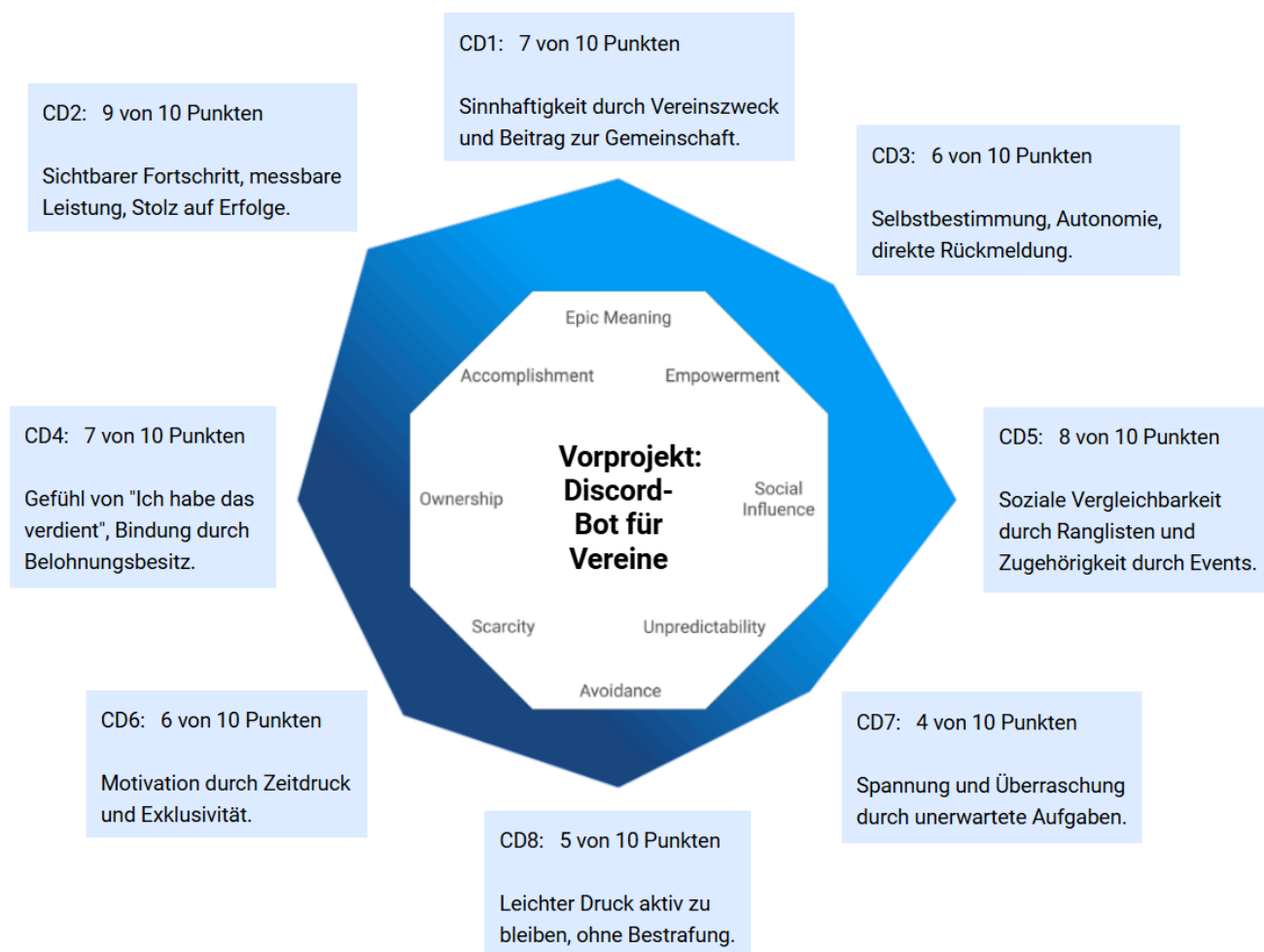
Das Octalysis Framework nach Yu-kai Chou (2015) ergänzt die psychologische Perspektive der Self-Determination-Theory (SDT) um ein praxisorientiertes Modell zur Analyse und Gestaltung von Motivation in digitalen Systemen. Es identifiziert acht sogenannte Core Drives, welche das Verhalten von Nutzern in Gamification-Umgebungen steuern. ([Chou, 2015](#))

Die acht Core Drives sind:

1. **Epic Meaning & Calling:** das Gefühl, Teil von etwas Grösserem zu sein.
2. **Development & Accomplishment:** Motivation durch Fortschritt und Belohnung.
3. **Empowerment of Creativity & Feedback:** Motivation durch kreative Freiheit und Rückmeldung.
4. **Ownership & Possession:** Motivation durch Besitz oder Kontrolle.
5. **Social Influence & Relatedness:** Motivation durch Gemeinschaft, Anerkennung oder Wettbewerb.
6. **Scarcity & Impatience:** Motivation durch Verknappung oder Limitierung.
7. **Unpredictability & Curiosity:** Motivation durch Neugier und Überraschung.
8. **Loss & Avoidance:** Motivation durch die Vermeidung negativer Konsequenzen.

Während die SDT die Qualität der Motivation (intrinsisch vs. extrinsisch) erklärt, zeigt das Octalysis-Framework konkrete Design-Hebel auf, mit denen diese Motivation im System gefördert werden kann. Für ClansCore wurde das Modell genutzt, um die im Vorprojekt umgesetzten Gamification-Elemente mit einer Bewertungs-Skala von 1 (noch gar nicht umgesetzt) bis 10 (vollumfänglich integriert) einzuschätzen und die Balance zwischen intrinsischen und extrinsischen Anreizen zu bestimmen.

Die Analyse der Vorarbeit zeigt eine klare Schwerpunktsetzung auf Development & Accomplishment (CD2) sowie Social Influence & Relatedness (CD5). Diese Drives erzeugen eine messbare Belohnungsstruktur und fördern sozialen Vergleich innerhalb des Vereins. Intrinsische Anteile sind vorhanden, aber weniger stark systemisch verankert.

**Abb. 1:** Die Bewertung des Vorprojektes im Octalysis-Modell

Core Drive		Einschätzung (Stand Vorprojekt)
CD1	Epic Meaning & Calling (7 von 10 Punkten)	Durch Rollen wie Mitglied oder Vorstand sowie den klar kommunizierten Vereinszweck entsteht ein Sinnbezug und Gemeinschaftsgefühl.
CD2	Development & Accomplishment (9 von 10 Punkten)	Das Punktesystem und die Ranglisten (saisonal, jährlich, allzeit) sind stark ausgeprägt. Sie erzeugen sichtbaren Fortschritt und messbaren Erfolge, die Hauptquelle extrinsischer Motivation.
CD3	Empowerment of Creativity & Feedback (6 von 10 Punkten)	Aufgaben können selbstständig übernommen und abgeschlossen werden. Rückmeldungen erfolgen durch Annahme oder Ablehnung seitens des Vorstands. Dies unterstützt Autonomie und Selbstwirksamkeit.
CD4	Ownership & Possession (7 von 10 Punkten)	Punkte und Belohnungen (Merchandise, Rabatte, Gratis-Mitgliedschaft) erzeugen ein Gefühl von Besitz und Zugehörigkeit.
CD5	Social Influence & Relatedness (8 von 10 Punkten)	Öffentliche Ranglisten sowie gemeinsame Events fördern Anerkennung und Verbundenheit in der Community.
CD6	Scarcity & Impatience (6 von 10 Punkten)	Zeitlich begrenzte Saisons und manuell vergebene Belohnungen erzeugen moderate Verknappung und fördern Aktivität innerhalb klarer Zyklen.
CD7	Unpredictability & Curiosity (4 von 10 Punkten)	Neben gelegentlichen, kurzfristig ausgeschriebenen Aufgaben fehlen explizite Zufalls- / Überraschungsmechaniken. Potenzial für Mystery-Belohnungen oder Wochen-Boosts sind vorhanden.
CD8	Loss & Avoidance (5 von 10 Punkten)	Ranglisten-Resets motivieren zur regelmässigen Teilnahme. Die Punkte selbst bleiben bestehen. Der Anreiz entsteht durch periodisch erneuerte Wettbewerbschancen ohne starken Druck.

Tab. 37: Die Bewertung des Vorprojektes im Octalysis-Modell

Das System adressiert insbesondere die extrinsisch motivierenden Core Drives Development & Accomplishment (CD2) und Ownership & Possession (CD4), welche durch das Punktesystem, Ranglisten und Belohnungen stark ausgeprägt sind. In Kombination mit Social Influence & Relatedness (CD5) entsteht eine klare, faire sowie gemeinschaftsorientierte Wettbewerbsdynamik.

Intrinsische Faktoren wie Epic Meaning & Calling (CD1) sind zwar über den Vereinszweck vorhanden, werden im Systemdesign jedoch nur indirekt vermittelt. Empowerment of Creativity & Feedback (CD3) zeigt erste Ansätze durch selbstständige Aufgabenübernahme und Vorstands-Feedback, bleibt aber im Potenzial begrenzt. Explorative und überraschende Elemente (CD7, Curiosity) sowie dynamische Risikofaktoren (CD8, Loss & Avoidance) sind bislang schwächer ausgeprägt.

Insgesamt ist der Discord-Bot in dieser Form primär extrinsisch und kompetenzorientiert motivierend, was Einstieg und Aktivierung stark unterstützt. Für eine nachhaltige, qualitativ hochwertige Motivation im Sinne der SDT sollte die Gewichtung intrinsischer Drives (insbesondere Empowerment, Curiosity, Meaning) erhöht werden, um Selbstbestimmung, Kreativität und persönliche Identifikation mit dem Vereinsleben langfristig zu fördern.

3.4 Erweiterungen, Chancen und Regeln

Auf Basis der Interview-Erkenntnisse (SDT) und der Octalysis-Analyse des Vorprojekts ergeben sich konkrete Erweiterungen, Chancen sowie Gestaltungs- und Fairnessregeln für ClansCore. Ziel ist es, die aktuell dominanten extrinsischen Treiber (CD2, CD4, CD5) um intrinsische Faktoren (CD1, CD3, CD7) zu ergänzen, ohne Fairness, Transparenz und Datenschutz zu gefährden.

3.4.1 Designprinzipien

Prinzip	SDT-Bezug	Octalysis-Hebel
Soziale Verbundenheit vor Wettbewerb	Relatedness: Anerkennung, Zugehörigkeit, Teamgefühl	• CD5: Social Influence • CD1: Meaning
Sichtbarer Fortschritt mit sinnvollem Kontext	Kompetenz: Feedback, Meilensteine, Beiträge „für den Verein“	• CD2: Development • CD4: Ownership
Selbstbestimmte Beteiligung statt Zwang	Autonomie: Wahlfreiheit, optionale Pfade	• CD3: Empowerment
Fairness & Datenschutz-by-Design	Alle drei Grundbedürfnisse nicht untergraben	• CD6: Scarcity • CD8: Loss (transparent)

Tab. 38: Leitprinzipien für ClansCore

3.4.2 Geplante Erweiterungen und Regeln

Die folgenden Erweiterungen sind so konzipiert, dass sie Autonomie (SDT) stärken und die Octalysis-Drives CD1/CD3/CD7 gezielt ausbalancieren. Die Wahl der Priorität ergibt sich aus der Wirkung und Umsetzbarkeit.

ID	Erweiterung	Motivationswirkung	Priorität
E1	Mehrstufiges Reminder-System für Events (Save-the-Date, Wochen-, Tages-Reminder, optional Direktnachricht). Optional: Export CSV / ICS im Dashboard (Kalendersubscribe).	- Beseitigt Kommunikations-Engpass - Steigert Teilnahme ohne Zwang (SDT: Kompetenz / Autonomie) - CD2 und CD5	Hoch
E2	Anerkennungssysteme in Form von speziellen Hervorhebungen (z. B. „Helfer des Monats“, Rollenfarben, öffentliche Danksagungen).	- Sichtbare Wertschätzung (SDT: Relatedness) - Ownership über sichtbare Symbole - CD1, CD4 und CD5	Hoch
E3	Datensparsame Vereins-Analytics im Dashboard (nur Aggregat aktiver Mitglieder, Eventquote, Helferquote und Trendlinien, ohne personenbezogenes Dauertracking).	- Kompetenz der Vorstände (Steuerungsfähigkeit) (SDT: Kompetenz) - Keine Untergrabung der Autonomie / Privacy (SDT: Autonomie) - CD2	Hoch
***	Punkte haben ein Ablaufdatum	CD8	Hoch
E4	Kooperative Formate wie Team-Quests, Gemeinschaftsziele (z. B. „50 Teilnahmen in 30 Tagen“) mit serverweiter Belohnung (z. B. Punktevergabe).	- Verlagerung von Wettbewerb zu Kooperation - Stärkt Verbundenheit & Sinn (SDT: Relatedness) - CD1, CD3, CD5	Mittel
E5	Event-Recaps im Kanal (kurzer Auto-Post mit Teilnehmerzahl, Dank an Helfer, optional mit Bild).	- Verstärkt Anerkennung und Sinn (SDT: Relatedness) - Erhöht Sichtbarkeit realer Beiträge - CD1, CD2 und CD5	Mittel
E6	Leichte Curiosity-Elemente wie zum Beispiel „Wochen-Boost“ (+20 % für genau eine Aktivität, transparent angekündigt) rotierend per Saison (kein Zufall bei Belohnungen).	- Erhöht Abwechslung ohne Intransparenz - Schützt Fairness - CD6 und CD7	Niedrig

Tab. 39: Erweiterungen und Zielwirkung

Die geplanten Regeln stellen Fairness, Datenschutz und langfristige Motivation sicher. Es übernimmt unter anderem die in der Vorarbeit bereits beschriebenen Regeln (vgl. [Vorprojekt, Kap. 4.4](#)) und formuliert sie mit den neuen Erkenntnissen implementationsnah für Discord-Bot und Dashboard.

Die Regeln lauten wie folgt:

- Keine Macht- oder Stimmenvorteile durch Punkte:** Verhindert toxische Hierarchien; erhält demokratische Vereinsprozesse (SDT: Relatedness / Autonomie).
- Punkte sind nicht übertragbar:** Verhindert Pay-to-Win / Transferhandel, schützt Fairness und Ownership.
- Saisonale Ranglisten-Resets, wobei Punkte erhalten bleiben:** Ergibt niedrige Einstiegshürden für Neumitglieder, da der Wettbewerb periodisch neu geöffnet wird (CD6 und CD8).
- Transparente Punktequellen und Limits:** Jede vergütete Aktivität ist dokumentiert. Limits an Aufgaben (täglich / monatlich) vermeiden übertriebenes Sammeln ohne fertigstellen.
- Datensparsame Analytics (nur Aggregat):** Steuerung ohne personenbezogenes Dauertracking sowie DSGVO- und Vereinskultur-konform.
- Freiwilligkeit:** Gamification ist optional und wird ohne Druck als zusätzliches Vereins-Angebot angesehen (SDT: Autonomie).

7. **Keine Überraschungs-Strafen:** Loss & Avoidance nur über Ranglisten-Resets, keine verdeckten Bestrafungen. Schützt intrinsische Motivation.
8. **Moderierte Curiosity:** Wochen-Boost oder Community-Quests sind angekündigt und fair verteilt. Keine Zufallsbelohnungen mit Ungleichheitseffekt.
9. **Angepasste Ästhetik und Sprache:** Gamification-Elemente sollen die Vereinsidentität widerspiegeln und nicht zu verspielt oder infantil wirken. Dadurch bleibt das System glaubwürdig und akzeptiert.

Bei der Weiterentwicklung von ClansCore ist darauf zu achten, dass Motivation nicht zu stark über Belohnungen gesteuert wird. Um eine Überbetonung extrinsischer Faktoren zu vermeiden, werden kooperative Formate und persönliche Zielpfade gezielt priorisiert. Ungleichheit durch Zufallsmechaniken wird ausgeschlossen, indem Curiosity-Elemente stets angekündigt und fair verteilt bleiben. Datenschutz wird durch strikte Aggregation der Kennzahlen gewährleistet.

3.4.3 Technische Implikationen und Roadmap

Kurzfristig werden die Erweiterungen E1 bis E3 umgesetzt, also das mehrstufige Reminder-System, neue Anerkennungsfunktionen sowie ein datensparsames Dashboard mit aggregierten Kennzahlen. Diese Massnahmen bilden den Kern der ersten Ausbaustufe und sollen sowohl die Beteiligung als auch die Sichtbarkeit von Engagement verbessern. Anschliessend werden, sofern ausreichend Zeit und Ressourcen vorhanden sind, die kooperativen Formate (E4) sowie automatisierte Event-Recaps (E5) in Angriff genommen. Beide Erweiterungen fördern Selbstbestimmung und Zusammenarbeit und dienen als nächste Entwicklungsstufe zur Stärkung intrinsischer Motivation. Das optionale Feature E7 (Curiosity-Elemente) gilt als Ergänzung, welches nach ersten Praxiserfahrungen schrittweise integriert werden könnte.

3.4.4 Zukünftige Erfolgsmessung

Eine direkte Messung der Wirksamkeit erfolgt im Rahmen dieser Arbeit nicht, kann aber künftig auf Basis anonymisierter Aggregatdaten erfolgen. Als hypothetische Indikatoren eignen sich Kennzahlen wie die Teilnahmequote an Events, der Anteil aktiver Helfer, die Anzahl vergebener Anerkennungen oder die Vielfalt der Aktivitäten pro Mitglied und Saison. Diese Grössen könnten in zukünftigen Auswertungen zeigen, ob sich Beteiligung, Sichtbarkeit und Fairness durch die neuen Mechanismen messbar verbessern. Die Datenerhebung soll dabei ausschliesslich aggregiert und ohne personenbezogene Speicherung erfolgen.

4 Erweitertes Sicherheitskonzept

Das erweiterte Sicherheitskonzept definiert die konzeptionellen und organisatorischen Massnahmen, mit denen die Vertraulichkeit, Integrität und Verfügbarkeit der in ClansCore verarbeiteten Daten gewährleistet werden. Es baut auf den im Vorprojekt etablierten Grundlagen zu Datensicherheit, Rechteverwaltung und Datenschutz (vgl. Vorprojekt, Kap. 3 Sicherheitskonzept) auf, erweitert diese jedoch in Hinblick auf die neue Systemarchitektur mit einem webbasierten Dashboard und der synchronisierten Datenhaltung zwischen Discord, der Webplattform (Dashboard) und potenziell weiteren Plattformen (UC11 , NFR6).

Das Backend, welches bereits im Vorprojekt als eigenständige, modulare Anwendung konzipiert wurde, bildet weiterhin das zentrale Bindeglied zwischen allen Plattformen. Neu hinzugekommen sind die Anforderungen an eine sichere Kommunikation zwischen Backend und Angular-Frontend sowie an eine konsistente und konfliktfreie Synchronisation von Benutzerdaten, Rollen und Gamification-Informationen über mehrere Zugriffspunkte. Die Systemarchitektur ist somit nicht nur funktional, sondern auch sicherheitstechnisch stärker verteilt, was eine Erweiterung des bisherigen Sicherheitsmodells erforderlich macht. (UC5 , UC8)

4.1 Ziel und Geltungsbereich

Ziel des erweiterten Sicherheitskonzepts ist es, die personenbezogenen und vereinsinternen Daten vor unbefugtem Zugriff, Manipulation und Verlust zu schützen. Der Fokus liegt auf Vertraulichkeit, Integrität und Verfügbarkeit, welche die drei zentralen Schutzziele der Informationssicherheit bilden (NIST, 2020) (NFR3 , NFR6 , NFR7).

Das Konzept erstreckt sich auf alle sicherheitsrelevanten Komponenten des Systems:

- **Backend** (Node.js / Express): zentrale Applikationslogik und API-Kommunikation
- **Discord-Bot** (Discord.js): Interaktionsschnittstelle für Mitgliederaktionen
- **Admin-Dashboard** (Angular): browserbasierte Verwaltungs- und Interaktionsplattform
- **Datenbank** (MongoDB): persistente Speicherung von Benutzer-, Rollen- und Aktivitätsdaten
- **Infrastruktur und Serverumgebung** (Docker, Linux-Host, GitHub Actions): Containerisierung, CI/CD und physische bzw. virtuelle Serverabsicherung
- **Externe Dienste** (Google Calendar API): Synchronisation von Kalendereinträgen

Nicht berücksichtigt werden die nativen Sicherheitsmechanismen externer Plattformen (Discord, Google), da sie ausserhalb der direkten Kontrolle des Systems liegen.

4.2 Sicherheitsprinzipien

Das Sicherheitskonzept orientiert sich an vier grundlegende Prinzipien der Informationssicherheit, welche als Fundament weiterer technischer und organisatorischer Sicherheitsmassnahmen dienen sowie in allen Systemschichten konsistent angewendet werden.

Prinzip	Beschreibung der Umsetzung
Least Privilege	Alle Komponenten und Benutzerrollen besitzen nur jene Rechte, die für ihre Funktion notwendig sind. Vorstandsmitglieder dürfen Daten auf Organisations-Ebene (Mitgliederverwaltung, Events, Punktesystem) bearbeiten, während Mitglieder ausschliesslich ihre eigenen Daten sehen und bearbeiten dürfen. Öffentliche Informationen wie Vereins-Events, Bewerbungsformular, Statuten und Datenschutzrichtlinie sind ohne Login zugänglich. (Bezug: <u>UC5</u>)
Defense in Depth	Sicherheit wird durch mehrere, voneinander unabhängige Schutzebenen gewährleistet, etwa durch Transportverschlüsselung, Container-Isolation, Zugriffsbeschränkungen und rollenbasierte Rechteprüfung. (<u>Murugiah & Karen, 2017</u> ; <u>NIST, 2020</u>) (Bezug: <u>NFR8</u>)
Accountability	Alle sicherheitsrelevanten Aktionen (Login-Versuche, Rollenänderungen, Punkteänderungen) werden protokolliert und sind für berechtigte Personen nachvollziehbar (Bezug: <u>NFR9</u>).
Privacy by Design	Es werden nur notwendige Daten gespeichert, analog zum im Vorprojekt definierten Datensparsamkeitsprinzip (vgl. <u>Vorprojekt</u> , Kap. 3.3 ff.). Die Datenschutzinformationen sind zusätzlich im Dashboard direkt einsehbar (Bezug: <u>NFR6</u>).

Tab. 40: Beschreibung der vier grundlegenden Prinzipien

4.3 Authentifizierung und Autorisierung

Die Authentifizierung erfolgt über mehrere, voneinander getrennte Ebenen:

- **Discord-OAuth2** für Nutzerinteraktionen innerhalb der Plattform (unverändert aus dem [Vorprojekt](#), vgl. [Kap. 3.2.1](#))
- **Dashboard-Authentifizierung** für den Webzugriff
- **Infrastruktur-Authentifizierung** für administrative Server- und Datenbankzugriffe

Das Backend fungiert dabei als zentrale Autorisierungsinstanz und verwaltet sämtliche Rollen- und Rechteinformationen. Es prüft eingehende Anfragen unabhängig von der Plattform, um konsistente Zugriffsbeschränkungen sicherzustellen.

Die Authentifizierung unterscheidet zwischen menschlichen Benutzer und automatisierten Dienstkonto. Menschliche Zugriffe erfolgen über eine interaktive Anmeldung und werden bei sensiblen Operationen durch eine [Multi-Faktor-Authentifizierung](#) ergänzt. Damit wird insbesondere der Zugriff auf Infrastruktur und Datenbanken gegen unautorisierte Übernahmen geschützt ([UC1](#) , [UC3](#) , [NFR6](#)).

4.3.1 Dienstkonto und automatisierte Zugriffe

Im Gegensatz zu menschlichen Benutzer greifen automatisierte Systemkomponenten (z. B. der Discord-Bot, das Backend oder der CI/CD-Prozess) über sogenannte Dienstkonto auf Systemressourcen zu. Diese Konto sind speziell für maschinelle Abläufe vorgesehen, arbeiten mit sicheren Tokens oder Secrets und unterliegen restriktiven Rechten nach dem Prinzip Least Privilege. Aktionen werden nachvollziehbar protokolliert (Bezug: [NFR9](#)) und Secrets werden sicher in der Pipeline verwaltet (Bezug: [NFR8](#)).

Bereits im [Vorprojekt](#) war der Discord-Bot als solches Dienstkonto umgesetzt. Er lief in einem eigenen Container, authenticizierte sich über ein Bot-Token aus Umgebungsvariablen (.env) und kommunizierte über die Discord-API mit dem System. Da der Bot umfängliche Berechtigungen benötigt, wird er weiterhin Administratorrechte im Discord-Server erhalten. Diese Ausnahme wird bewusst akzeptiert, da sie für die technische Funktionsfähigkeit des Systems erforderlich ist.

Die Trennung zwischen menschlichen Admins (mit MFA-geschütztem Zugriff) und automatisierten Systemprozessen bleibt dennoch gewahrt. Dienstkonto werden systemweit durch [Role-Based Access Control](#) (nachfolgend RBAC genannt) begrenzt und besitzen nur die für ihren Anwendungsfall notwendigen Berechtigungen. Dadurch wird das Risiko einer Kompromittierung oder eines Missbrauchs erheblich reduziert.

4.3.2 Absicherung des Datenbankzugriffs

Die Datenbank (nachfolgend DB genannt) stellt das sicherheitskritischste Element des Systems dar, da sie sämtliche personenbezogenen und transaktionsbezogenen Daten des Vereins enthält. Entsprechend muss der Zugriff sowohl technisch als auch organisatorisch auf mehreren Ebenen geschützt werden. Die Absicherung umfasst einerseits systeminterne Verbindungen zwischen Backend, Bot und DB, andererseits den administrativen Zugang für menschliche Benutzer mit erweiterten Berechtigungen (Administratoren).

Im Rahmen der Konzeptentwicklung wurden drei Ansätze zur MFA des DB-Zugriffs untersucht. Hintergrund ist, dass MongoDB selbst keinen nativen Zwei-Faktor-Login unterstützt. Der zweite Faktor muss daher vorgelagert, also auf Netzwerkebene oder durch Besitzfaktoren wie Zertifikate, umgesetzt werden.

4.3.2.1 Bewertung und Entscheid

Die Analyse zeigt, dass Netzwerk-MFA den besten Kompromiss zwischen Sicherheit, Wartbarkeit und Implementierbarkeit bietet. Sie schützt den gesamten Zugriffspfad zur Datenbank und kann ohne strukturelle Änderungen an bestehenden Komponenten eingeführt werden. In der bestehenden OST-Serverumgebung (virtuelle Linux-Server mit SSH-Zugang) lässt sich diese Methode schnell, kostengünstig und mit geringem Risiko implementieren. Darüber hinaus ist sie vollständig kompatibel mit der bestehenden Container- und CI/CD-Umgebung (Docker, GitHub Actions).

Die detaillierte Evaluation und der technische Vergleich der einzelnen Varianten (Netzwerk-MFA, X.509-Zertifikate, SSO / OIDC) sind im Anhang dokumentiert. (siehe Anhang [Abschnitt 15.1](#))

4.3.2.2 Übertragbarkeit auf andere Vereine

Das vorgestellte Sicherheitskonzept berücksichtigt, dass ClansCore auch von Vereinen betrieben wird, die ihre Infrastruktur eigenständig hosten. Viele kleinere Organisationen verfügen über keinen dedizierten IT-Betrieb und müssen Sicherheit daher mit möglichst geringen Kosten und administrativem Aufwand gewährleisten. Für solche Vereine stellt das Modell der Netzwerk-MFA eine besonders praktikable Lösung dar. Es erfordert weder teure Identity-Management-Lösungen noch komplexe Zertifikatsverwaltungs-Systeme und kann mit frei verfügbaren Open-Source-Werkzeugen (z. B. OpenVPN¹) umgesetzt werden. Dadurch bleibt das Sicherheitsniveau hoch, ohne dass der Betrieb zusätzliche finanzielle oder organisatorische Hürden verursacht. Diese Skalierbarkeit ist zentral, um den Einsatz von ClansCore auch in kleineren, ehrenamtlich geführten Vereinen langfristig sicherzustellen ([UC11](#) , [NFR6](#)).

4.3.3 Dashboard-Login

Das Admin-Dashboard verfügt über eine eigenständige Authentifizierungsebene, die eine sichere Verwaltung von Vereinsdaten unabhängig von Discord ermöglicht. Die Benutzerverwaltung erfolgt zentral im Backend, welches Authentifizierung und Autorisierung des Web-Frontends übernimmt.

Nach erfolgreicher Anmeldung erhalten Nutzer eine kurzlebige Sitzungsberechtigung, die an ihre Rolle (Admin, Vorstand, Mitglied, Gast) gebunden ist. Rollenänderungen oder sicherheitsrelevante Ereignisse führen zum sofortigen Widerruf aktiver Sitzungen.

Die Berechtigungsprüfung erfolgt ausschliesslich serverseitig über rollenbasierte Zugriffskontrolle ([RBAC](#)). Alle Sitzungen und Aktionen werden im Audit-Log erfasst, um Nachvollziehbarkeit und Integrität sicherzustellen ([NFR9](#)). Für privilegierte Konten (z. B. Vorstand) ist eine optionale MFA vorgesehen ([NFR6](#)). Das Dashboard unterstützt zudem sichere Passwortverwaltung und Wiederherstellung ([UC3](#) , [UC4](#)). Technische Details zu den Sicherheitsmechanismen sind im Anhang dokumentiert. (siehe Anhang [Abschnitt 15.2](#))

Diese Umsetzung kombiniert Benutzerfreundlichkeit mit hoher Sicherheit und folgt den Prinzipien Least Privilege, Defense in Depth und Accountability.

4.4 Daten- und Schnittstellensicherheit

Die Kommunikation zwischen den Systemkomponenten erfolgt ausschliesslich über gesicherte Kanäle:

Verbindung	Technologie	Absicherung
Dashboard ↔ Backend	REST-API / HTTPS	TLS 1.3 + Token-basierte Auth (JWT im HttpOnly-Cookie)
Backend ↔ Discord-Bot	WebSocket Event-Bridge	Bot-Token + Signaturprüfung
Backend ↔ MongoDB	TCP / TLS	SCRAM-SHA-256 + RBAC + Netzwerk-MFA
Backend ↔ Google API	OAuth2 Client-Secrets	Zugriffstoken mit Scopes

Tab. 41: Beschreibung der abgesicherten Kanäle in ClansCore

4.4.1 Eingabevalidierung

Alle externen Eingaben (REST, WebSocket, Formular) werden serverseitig validiert und sanitisiert (Injection-Schutz). Fehler werden kontrolliert behandelt und geloggt, ohne sensitive Details preiszugeben (Bezug: [NFR3](#) , [NFR9](#)).

4.4.2 Synchronisierung und Konfliktlösung

Da ClansCore sowohl über den Discord-Bot als auch über das Admin-Dashboard oder anderen potentiellen Plattformen genutzt wird, können Änderungen an denselben Datensätzen parallel erfolgen. Beispielsweise kann ein Vorstandsmitglied ein Mitgliederprofil im Dashboard anpassen, während der Bot gleichzeitig eine Punktebuchung durchführt. Um daraus entstehende Datenkonflikte zu vermeiden, wird im Backend eine optimistische Nebenläufigkeitskontrolle (Optimistic Concurrency Control, OCC) angewendet. Eine detaillierte Beschreibung des Transaktions- und Konfliktlösungsverfahrens findet sich im Anhang. (siehe Anhang [Abschnitt 15.3](#))

¹<https://openvpn.net/>

Dieses Verfahren stellt sicher, dass Datenintegrität und Nachvollziehbarkeit auch bei paralleler Nutzung durch mehrere Plattformen gewährleistet bleiben. Es basiert auf anerkannten Architekturmustern zur Transaktions- und Konsistenzsicherung in verteilten Systemen ([Martin, 2003](#)) (Bezug: [NFR3](#) , [NFR9](#)).

4.5 Datenschutz und DSGVO / DSGVO-Umsetzung

Die im Vorprojekt formulierten Datenschutzrichtlinien (vgl. [Vorprojekt, Kap. 3.3 - 3.5](#)) gelten unverändert.

Neu können Nutzer direkt im Dashboard ihre gespeicherten Informationen einsehen, die Einwilligung zur Datenspeicherung widerrufen oder eine Datenlöschung anfordern. Das Hosting erfolgt auf Servern in der Schweiz ohne US-Rechtsbezug, wodurch die Datenhoheit gewahrt bleibt und der Zugriff durch ausländische Behörden (z. B. über den CLOUD Act) ausgeschlossen ist ([U.S.-Department-of-Justice, 2019](#)) . Die Verarbeitung personenbezogener Daten richtet sich nach den Grundsätzen der DSGVO ([Europäische-Union, 2016](#)) , welche Privacy by Design and by Default voraussetzen, sowie dem schweizerischen Datenschutzgesetz ([Schweizerische-Eidgenossenschaft, 2020](#)) . (Bezug: [NFR6](#))

4.6 Server- und Deployment-Sicherheit

Das Backend wird in einer containerisierten Umgebung betrieben, die nach dem Prinzip Defense in Depth aufgebaut ist. Ziel ist eine mehrschichtige Absicherung, bei der einzelne Schutzebenen unabhängig voneinander wirken. ([NIST, 2020](#))

Weitere technische Details zur Container-Isolation, Namespaces, Capability-Drops und Netzwerksegmentierung sind im Anhang erläutert. (siehe Anhang [Abschnitt 15.4](#))

Durch die Kombination von Container-Isolation, Netzwerksegmentierung und Multi-Faktor-Authentifizierung entsteht eine robuste, mehrschichtige Sicherheitsarchitektur, welche die Vertraulichkeit, Integrität und grundlegende Verfügbarkeit des Systems sicherstellt. Eine umfassende Gewährleistung der Verfügbarkeit würde jedoch zusätzliche Massnahmen erfordern, wie beispielsweise eine Systemverteilung (Distribution), Lastverteilung (Load Balancing) oder automatisierte Failover-Mechanismen. Diese Aspekte gehen über den Rahmen der vorliegenden Arbeit hinaus und werden daher nicht weiter berücksichtigt. (Bezug: [NFR8](#) , [NFR6](#))

4.7 Manipulations- und Integritätsschutz

Wie in der Vorarbeit beschrieben (vgl. [Vorprojekt, Kap. 3.3](#)) werden alle Punkte- und Event-Transaktionen manipulations-sicher in der Datenbank protokolliert. Neu ergänzt sind:

- **Transaktionale Speicherung** mittels atomarer Operationen wie MongoDB Sessions ([MongoDB-Contributors, 2025a](#)) .
- **Audit-Log** aller Änderungen, insbesondere von Gamification-Daten, sichtbar für den Vorstand.
- Optionaler **Hash-Check** für Integritätsprüfung einzelner Datensätze.

Damit bleibt jede Änderung rückverfolgbar und das Vertrauen in die Datenbasis langfristig erhalten. (Bezug: [NFR7](#) , [NFR9](#))

4.8 Risikoanalyse und Massnahmen

Die zentrale Sicherheitsrisiken umfassen unautorisierte Zugriffe, Datenlecks, Social-Engineering-Angriffe und Datenverlust. Für jedes Risiko existieren die folgenden präventiven und reaktiven Massnahmen.

Risiko	Auswirkungsgrad	Massnahme	Status
Datenleck durch API	Hoch	HTTPS, Token-Auth, Input-Validation (NFR6 , NFR3)	Offen
Unautorisierter DB-Zugriff	Hoch	Netzwerk-MFA + SCRAM-SHA-256 + RBAC (NFR6 , NFR9)	Offen
Social Engineering	Mittel	Schulung, Prinzip der minimalen Rechte (UC5)	Offen
Datenverlust	Hoch	Regelmässige Backups + Restore-Tests	Offen

Tab. 42: Einschätzung der Risiken und deren Massnahmen

4.9 Monitoring und Wartung

Ein konsistentes Logging-Konzept dokumentiert sicherheitsrelevante Ereignisse (Auth-Flows, Rollenänderungen, fehlgeschlagene Zugriffe, Konfliktauflösungen). Für schwerwiegende Vorkommnisse sind Benachrichtigungen an einen Discord-Admin-Kanal und das Admin-Dashboard vorgesehen. Wiederkehrende Sicherheitsüberprüfungen orientieren sich an der OWASP-Top-10-Checkliste ([OWASP-Contributors, 2021](#)) . Konfigurations- und Rechteprüfungen erfolgen regelmässig. (Bezug: [NFR9](#))

Diese Massnahmen schaffen einen kontinuierlichen Überblick über die Systemintegrität und ermöglichen ein rasches Eingreifen bei Sicherheitsvorfällen.

4.10 Abgrenzung des Sicherheitskonzeptes

Das vorliegende Sicherheitskonzept fokussiert auf zentrale sicherheitsrelevante Anforderungen, die im Rahmen dieser Arbeit spezifiziert, implementiert und überprüft werden können. Weitere Massnahmen, wie erweiterte Schutzmechanismen für API-Rate-Limiting, Security-Header, Key-Rotation oder detaillierte Frontend-Härtung, werden als Best Practices betrachtet, jedoch nicht explizit dokumentiert oder umgesetzt, da deren Implementierung stark von der jeweiligen Hosting- und Vereinsumgebung abhängt.

Diese Themen werden im Projekt dann adressiert, wenn sie technisch erforderlich sind oder sich aus zukünftigen Erweiterungen ergeben. Die bewusste Fokussierung auf Authentifizierung, Zugriffskontrolle, Datenintegrität und Datenschutz stellt sicher, dass die sicherheitskritischen Kernanforderungen ([NFR3](#) , [NFR6](#) , [NFR9](#)) erfüllt und überprüfbar bleiben, ohne die Arbeit mit Randaspekten zu überfrachten.

5 Systemanalyse, Architektur und Wireframes

In diesem Kapitel werden die grundlegenden Strukturen und technischen Konzepte des Systems beschrieben. Zunächst wird anhand einer Domänenanalyse modelliert, wie die organisatorischen Abläufe, Rollen und Funktionen eines Vereins digital abgebildet werden können. Darauf aufbauend folgt die Darstellung der Systemarchitektur nach dem C4-Modell. Zum Schluss kommen die Wireframes mit den geplanten Ansichten im Dashboard.

5.1 Domain Analyse

Die Domain-Analyse baut auf dem im Vorprojekt entwickelten Vereins-Bot auf, der bereits zentrale Verwaltungs- und Gamification-Funktionen vereinte. Der Bot unterstützte die Mitgliederaufnahme, Rollenverwaltung sowie die Organisation von Aufgaben und Events. Gleichzeitig förderte er durch ein transparentes Punktesystem Motivation und Engagement im Vereinsalltag. (Alves & Schnider, 2025)

Im Zentrum der Domäne steht weiterhin die Person, die als Mitglied oder Vorstand mit dem System interagiert. Bekannte Entitäten wie Aufgabe, Event, Transaktion, Belohnung und Rangliste bilden den Kern des Vereinsworkflows. Mitglieder erfüllen Aufgaben, nehmen an Events teil oder leisten Spenden und erhalten dafür Punkte, die in Belohnungen eingelöst werden können. Ergänzend bleiben Verwaltungsprozesse wie Mitgliederverwaltung und Rollenvergabe zentrale Bestandteile des Systems.

Mit ClansCore wird diese Grundlage zu einem umfassenden Vereinsmanagement-System weiterentwickelt. Neu eingeführt werden ein plattformunabhängiges Account-System (PlattformAccount), das den bisherigen Discord-Bezug ablöst, die Verwaltung gesamthaft zentralisiert und vereinfacht (Kein direkter Datenbank-Zugriff) sowie Erinnerungsfunktionen für Events und Zahlungen, beispielsweise für jährliche Mitgliederbeiträge, hinzufügt. Die Nutzung der Plattformen bleibt dabei für Mitglieder optional. Darüber hinaus erweitert das System Transaktionen um zeitlich begrenzte Punkte (PunkteAblaufDatum), wodurch saisonale Resets und dynamische Belohnungsregeln ermöglicht werden.

Insgesamt entwickelt sich ClansCore damit von einem Discord-basierten Bot zu einem integrierten, plattformunabhängigen Vereinsökosystem, das Verwaltung, Gamification und Datenanalyse vereint und die Grundlage für eine transparente und nachhaltige digitale Vereinsführung schafft.

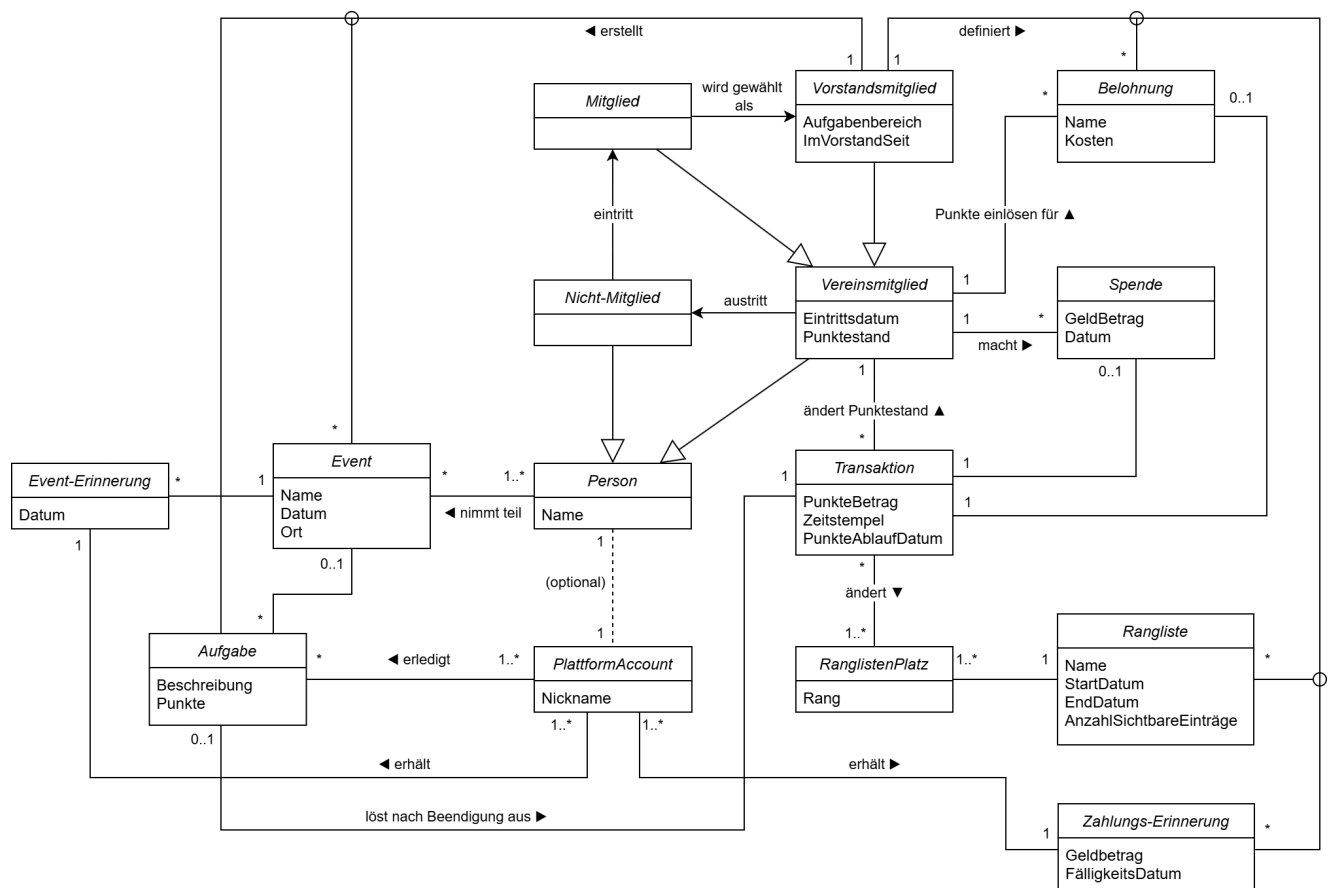


Abb. 2: Domain Model

5.2 C4-Modell

Die Architektur wurde mit dem C4-Modell ausgearbeitet, um eine klare, nachvollziehbare Struktur und eine nachhaltige Erweiterbarkeit zu gewährleisten. Bereits im Vorprojekt wurde der Discord-Bot modular konzipiert, die Entkopplung der Komponenten konnte jedoch noch nicht vollständig umgesetzt werden. Die dafür notwendige Grundstruktur ist im bestehenden Code vorhanden, erfordert aber ein Refactoring, um die Trennung von Verantwortlichkeiten konsequent zu realisieren und zukünftige Erweiterungen zu vereinfachen.

Das nachfolgende Architekturkonzept zeigt den geplanten Zielzustand des Systems. Es wurde so entworfen, dass die Kommunikationskanäle (Discord-Bot und Dashboard) in Zukunft auch durch andere Clients (z. B. Microsoft Teams) ersetzt oder ergänzt werden kann. Ebenso ist der Kalender-Provider abstrahiert, sodass alternative Dienste neben dem derzeit eingesetzten Google Calendar eingebunden werden können.

5.2.1 System Context Diagramm

ClansCore bildet das zentrale System des Vereins und stellt die gemeinsame Basis für alle angeschlossenen Plattformen dar. Über eine einheitliche API können sowohl der Discord-Bot als auch das neu hinzugefügte Web-Dashboard auf dieselbe Logik und Datenbasis zugreifen. Beide Kommunikations- und Verwaltungs-Kanäle interagieren über HTTPS mit der ClansCore-API.

Externe Dienste wie der Google Calendar werden zur Synchronisation von Events eingebunden. Die Architektur ist so ausgelegt, dass weitere Plattformen oder Kalender-Provider künftig ohne strukturelle Änderungen ergänzt werden können.

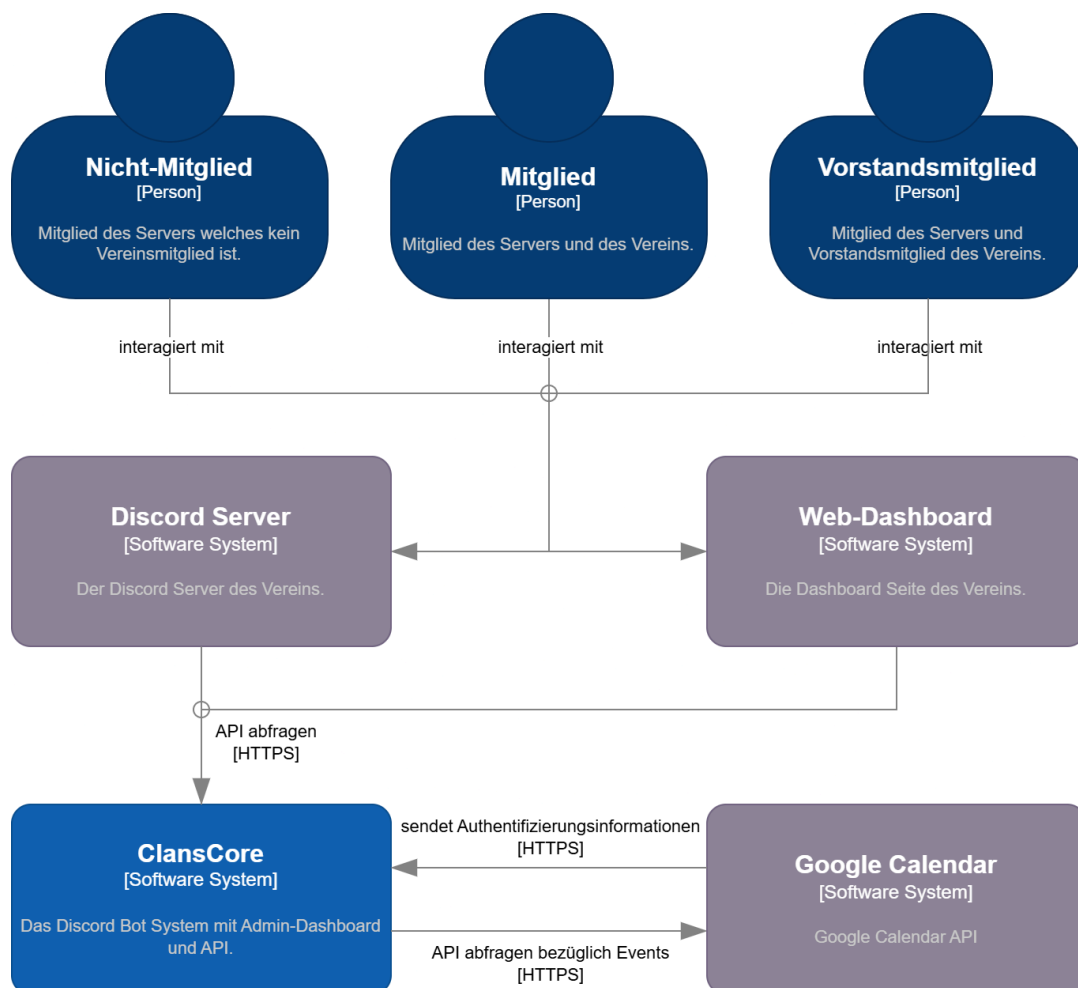


Abb. 3: System Context Diagramm

5.2.2 Container Diagramm

Das ClansCore System besteht aus den zwei zentralen Containern API-Applikation und Datenbank. Die API-Applikation ist in TypeScript auf Basis von Node.js implementiert und dient als Kommunikationsschnittstelle zwischen den Plattformen (Discord und Web-Dashboard) sowie der persistenten Datenhaltung. Sie verwaltet die Geschäftslogik, authentifiziert Anfragen und steuert den Datenaustausch mit externen Diensten wie dem Google Calendar.

Die Datenbank basiert auf MongoDB und speichert alle relevanten Informationen zu Nutzern, Rollen, Punkten, Aufgaben und Events. Über eine klar definierte Repository-Struktur wird eine saubere Trennung zwischen Datenzugriff und Anwendungslogik gewährleistet.

Discord-Server und Web-Dashboard greifen über HTTPS auf die API zu, um Daten zu lesen oder Aktionen auszuführen. Externe Systeme wie der Google Calendar werden ebenfalls über standardisierte API-Endpunkte eingebunden. Dadurch bleibt das System modular, erweiterbar und unabhängig von einzelnen Plattformen oder Providern.

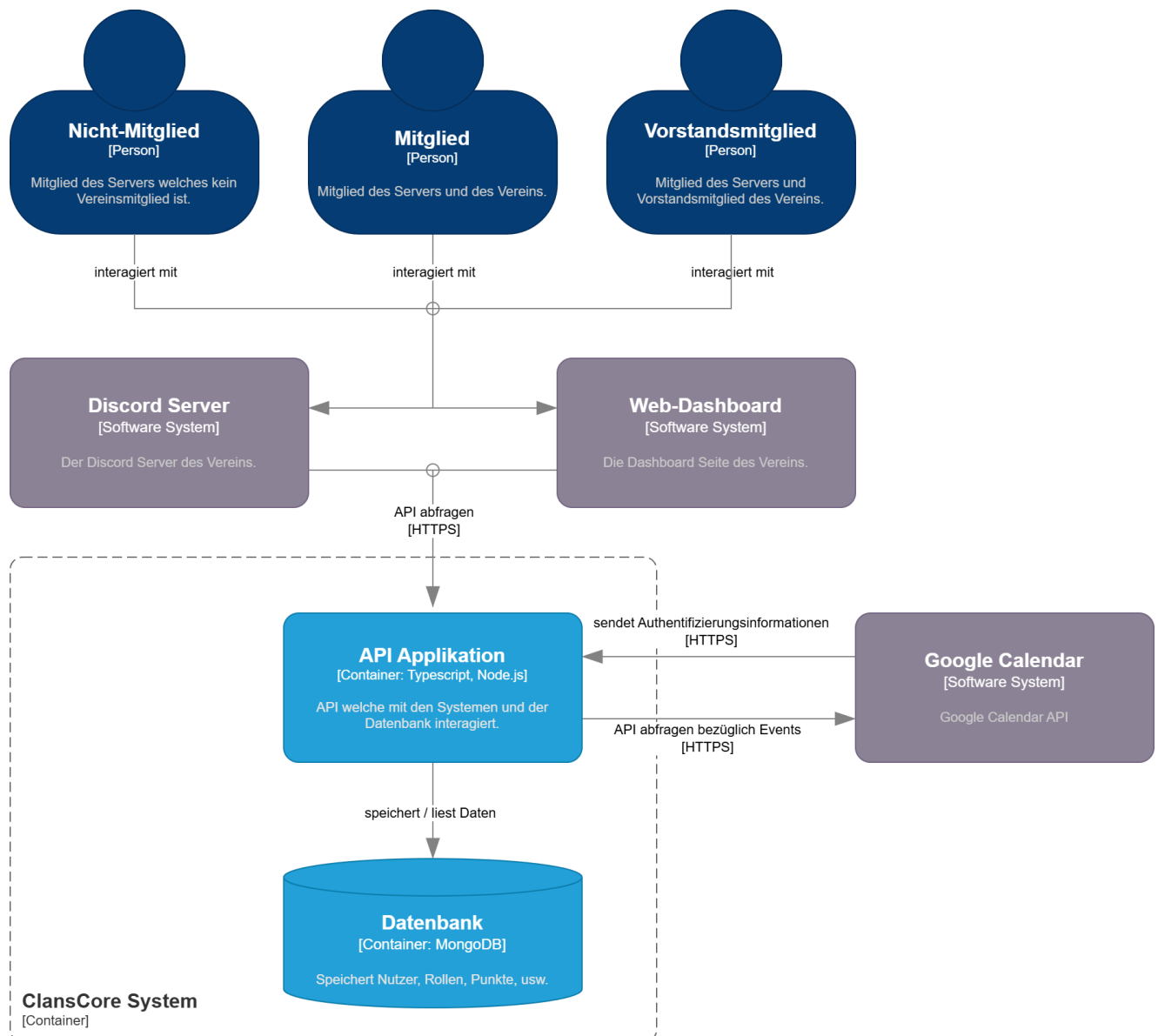


Abb. 4: Container Diagramm

5.2.3 Component Diagramm

5.3 Wireframes

Zur Skizzierung des Aufbaus und der Struktur der ClansCore-Webseite wurde das Wireframe-Tool Balsamiq verwendet. Die Wireframes wurden unter Bezug auf bestehende Angular-Webseiten entworfen ([madewithangular-Contributors, 2025](#)) und mit Blick auf die Nutzung von Angular-Material-Komponenten konzipiert. Es wurden nicht für alle Seiten Wireframes erstellt, da diese in erster Linie dazu dienen, einen allgemeinen Überblick über die Gestaltung und Funktionalität des Web-Dashboards zu vermitteln.

5.3.1 Nicht-Mitglied

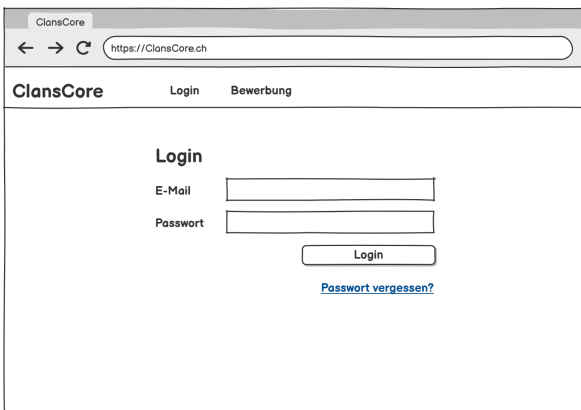


Abb. 5: Login

Login

Um die Kernfunktionen von ClansCore nutzen zu können, ist eine Authentifizierung erforderlich. Diese erfolgt auf der Login-Seite durch die Eingabe der E-Mail-Adresse sowie des zugehörigen Passworts ([UC1](#)). Für den Fall, dass ein Benutzer sein Passwort vergessen hat, besteht über einen entsprechenden Link die Möglichkeit, den Passwort-Zurücksetzungsprozess zu starten und ein neues Passwort anzufordern ([UC3](#)).

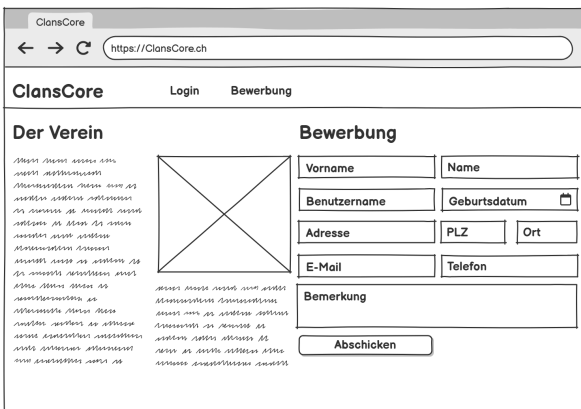
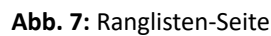


Abb. 6: Bewerbungsformular

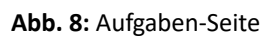
Bewerbung

Die Bewerbungsseite ist zweispaltig aufgebaut. Auf der linken Seite werden allgemeine Informationen über den Verein präsentiert, während auf der rechten Seite ein Formular zur Einreichung einer Beitrittsbewerbung zur Verfügung steht. ([UC2](#))

Mitglied



Auf der Ranglisten-Seite befindet sich auf der linken Seite eine filterbare Rangliste, die einen Überblick über die Platzierungen der Nutzer bietet. Auf der rechten Seite werden die eigenen Punkte, deren Historie und aktuelle Belohnungen angezeigt. (UC8)



Auf der Aufgaben-Seite werden alle verfügbaren Aufgaben angezeigt. Diese lassen sich nach verschiedenen Kriterien filtern (...). Solange eine Aufgabe noch nicht abgeschlossen ist, besteht die Möglichkeit, sich dafür anzumelden. (UC9) Vorstandsmitglieder haben zusätzlich die Berechtigung, Aufgaben zu bearbeiten und hinzuzufügen. (UC7)

Abb. 9: Mitglieder-Seite

Zugriff auf die Mitglieder-Seite haben ausschliesslich Vorstandsmitglieder. Es können bestehende Mitglieder gefiltert, bearbeitet, hinzugefügt oder gelöscht werden. (UC5)

5.3.2 Gamification-Verwaltung

In der Gamification-Verwaltung werden alle Parameter welche für die Punkteberechnung von Aufgaben benötigt werden erfasst. Da die Gamification-Verwaltung ein bestehendes Excel-Tool aus dem Vorprojekt (Alves & Schnider, 2025) ablöst, wurde das Design gezielt an dessen Layout und Bedienlogik angelehnt. Dadurch soll der Übergang zur neuen Anwendung für die Benutzer erleichtert werden.

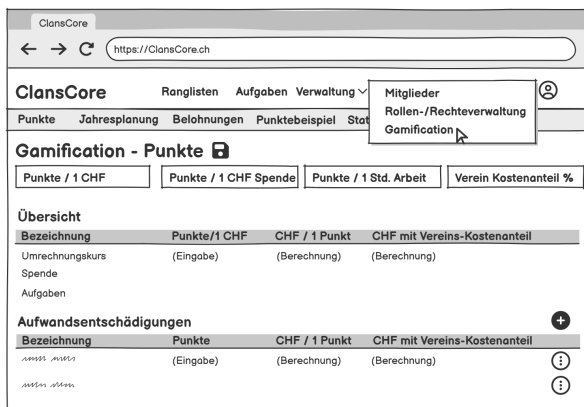
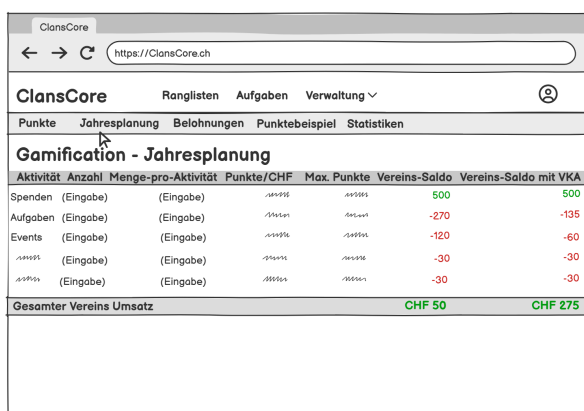


Abb. 10: Punktedefinitions-Seite

Punktedefinition

Auf der Punkte-Seite werden die zentralen Parameter des Gamification-Systems vom Vorstand definiert. Dazu gehören unter anderem der monetäre Wert eines Punktes, die Punktbewertung einer Arbeitsstunde sowie der Punktwert einer Spende. Zusätzlich kann der Kostenanteil des Vereins festgelegt werden. Darüber hinaus besteht die Möglichkeit, neue Aufwandsentschädigungen anzulegen, bestehende zu bearbeiten oder zu löschen. (UC6)



Aktivität	Anzahl	Menge-pro-Aktivität	Punkte/CHF	Max. Punkte	Vereins-Saldo	Vereins-Saldo mit VKA
Spenden (Eingabe)	(Eingabe)				500	500
Aufgaben (Eingabe)	(Eingabe)				-270	-135
Events (Eingabe)	(Eingabe)				-120	-60
Aufwandsentschädigungen (Eingabe)	(Eingabe)				-30	-30
Aufwandsentschädigungen (Eingabe)	(Eingabe)				-30	-30
Gesamter Vereins Umsatz					CHF 50	CHF 275

Abb. 11: Jahresplanung-Seite

Jahresplanung

Auf der Jahresplanung-Seite können die jährlichen Kosten für den Verein simuliert werden. Zur Berechnung werden Parameter von der Punkte-Seite benötigt. Dadurch lässt sich abschätzen, wie sich unterschiedliche Szenarien auf den jährlichen Gesamtumsatz des Vereins auswirken. (UC6)

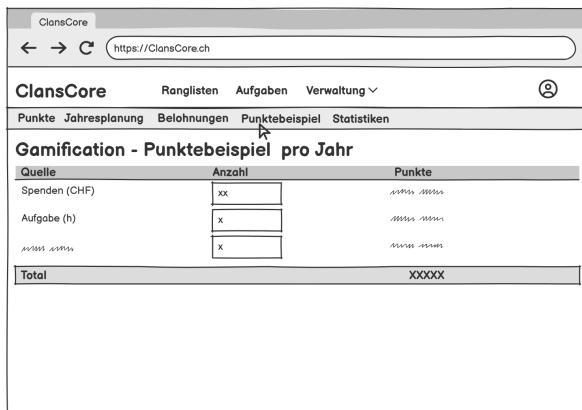


Belohnung	Realwert (CHF)	Ziel-Kostenanteil (%)	Ziel-Kostenanteil (CHF)	Punktwert	Aufwand (h)

Abb. 12: Belohnung-Seite

Belohnungen

Es können Belohnungen bearbeitet, hinzugefügt oder gelöscht werden. Zur Berechnung des Punktwertes und dem Aufwand werden Parameter von der Punkte-Seite benötigt. (UC6)



Quelle	Anzahl	Punkte
Spenden (CHF)	xx	10000
Aufgabe (h)	x	10000
Anzahl	x	10000
Total		XXXXX

Abb. 13: Punktebeispiel-Seite

Punktebeispiel

Auf der Punktebeispiel-Seite kann die Punktevergabe für ein Mitglied simuliert werden. Dabei werden verschiedene Aktivitäten, wie Spenden, Event-Teilnahmen oder Aufgaben, berücksichtigt, um die resultierende Gesamtpunktzahl zu berechnen und deren Auswirkungen im Punktesystem zu veranschaulichen. ([UC6](#))

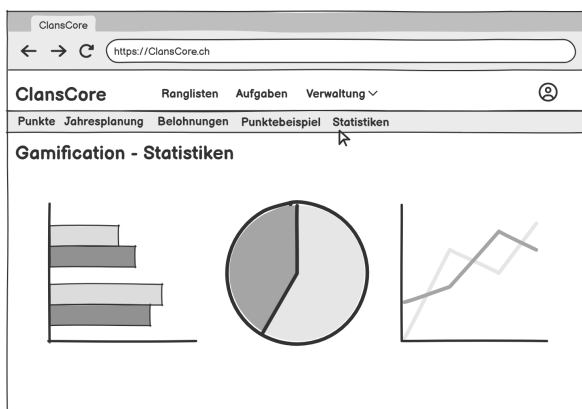


Abb. 14: Statistiken-Seite

Statistiken

Auf der Statistik-Seite können Auswertungen zu Events und Mitgliedern angezeigt werden. Diese bieten einen Überblick über relevante Kennzahlen und zukünftige Trends. Diese unterstützen den Vorstand bei der Analyse der Vereinsaktivitäten und dient als Planungshilfe. ([UC8](#))

6 Qualitätssicherung

Die Qualitätssicherung spielt eine entscheidende Rolle um eine stabile, sichere und fehlerfreie Anwendung bereitzustellen. In diesem Kapitel werden die Teststrategien beschrieben, sowie erläutert wie die Code-Qualität sichergestellt werden kann. Ziel ist es durch Tests und Überprüfungen eine hohe Softwarequalität zu gewährleisten.

6.1 Testkonzept

Das Testkonzept beschreibt die Vorgehensweise zur Sicherstellung der Software-Qualität für den Discord-Bot. Es legt fest, welche Tests durchgeführt werden, wie diese organisiert und dokumentiert werden und welche Tools und Umgebungen dabei zum Einsatz kommen. Ziel ist es, die Einhaltung der funktionalen und nicht-funktionalen Anforderungen sicherzustellen und eine fehlerfreie Anwendung zu gewährleisten.

6.1.1 Vorgehensweise

Um den Erfolg bei dem Testen sicherzustellen ist es wichtig strukturiert vorzugehen. Die Vorgehensweise lautet wie folgt:

1. Mittels der Requirements Tests definieren.
2. Entscheiden wie die Tests durchgeführt werden.
3. Tests durchführen.
4. Fehler und Erfolge dokumentieren.
5. Aufgefallene Fehler, die während den Testen auftauchen, beheben.
6. Tests erneut durchführen.

6.1.2 Testziele

Folgende Elemente sollen getestet werden:

- Alle Use Cases
- Alle Non-Functional Requirements
 - Usability
 - Maintainability
 - Reliability
 - Compatibility
 - Security
 - Performance

6.1.3 Testumgebung

Noch nicht definiert.

6.1.4 Unit- und Integrationstests

Die Unit- und Integrationstests des Admin-Dashboards werden mit dem Jasmine-Testing-Framework implementiert, das standardmässig in Angular-Projekten enthalten ist. Die Ausführung der Tests erfolgt über den Karma Test Runner, welcher als Laufzeitumgebung für automatisierte Tests dient und eine Integration in den Entwicklungsprozess ermöglicht. ([molily, 2025](#))

6.1.5 Usability-Tests

Um die Benutzerfreundlichkeit und Funktionalität des Admin-Dashboard zu prüfen, werden Usability-Tests mit mindestens Drei Testpersonen durchgeführt. Die Tester werden aufgefordert, verschiedene Funktionen und Abläufe durchzuführen. Die Tests werden mithilfe eines Formulars durchgeführt, welches den Testern die Aufgaben beschreibt. Diese können sie nach der Durchführung bewerten. Bei der Bewertung wird die Klarheit der Durchführung sowie auch der Nutzen der Funktion bewertet.

6.2 Ablaufplan

Der Ablaufplan beschreibt die verschiedenen Testphasen und deren Durchführung. Die Ergebnisse der Tests liefern wertvolle Informationen zur Stabilität, Funktionalität und Benutzerfreundlichkeit des Systems.

ID	T1
Name	Unit-Tests
Beschreibung	Funktionen im Backend werden mit Unit-Tests getestet. Die Tests werden nach jedem Commit automatisch durch GitLab-Pipelines durchgeführt.
Methode	Automatisch
Relevant für	Use-Cases, Functional Requirements
Testumgebung	
Intervall	Bei jedem Commit

Tab. 43: Ablauf Unit-Tests

ID	T2
Name	Integration-Tests
Beschreibung	Bei den Integration-Tests wird überprüft, ob die Verbindungen der verschiedenen Komponenten richtig funktionieren.
Methode	Automatisch
Relevant für	Use-Cases, Functional Requirements
Testumgebung	
Intervall	Bei jedem Commit

Tab. 44: Ablauf Integration-Tests

ID	T3
Name	System-Tests
Beschreibung	Am Ende wird für das gesamte System überprüft, ob alle <u>Use Cases</u> und <u>Non-Functional Requirements</u> erfolgreich umgesetzt worden sind.
Methode	Manuell
Relevant für	Alle Testziele
Testumgebung	
Intervall	Am Ende

Tab. 45: Ablauf System-Tests

ID	T4
Name	Usability-Tests
Beschreibung	Das Admin-Dashboard wird, wie im Abschnitt <u>Usability-Test</u> genauer beschrieben, von mehreren Testern getestet.
Methode	Manuell
Relevant für	Usability, Security, Requirements, Reliability
Testumgebung	
Intervall	Gegen Ende der Konstruktionsphase

Tab. 46: Ablauf Usability-Test

6.3 Code-Qualität

Um die Code-Qualität sicherzustellen wird das Vier-Augen-Prinzip angewendet. Bei jedem Meeting wird der geschriebene Code von den Teammitgliedern überprüft bevor er in den Main-Branch gemerged wird. Weitere Informationen zu Code-Qualität und GitHub können im [Abschnitt 11](#) gefunden werden.

Um die Einheitlichkeit des Codes sicherzustellen wird ein Linter verwendet. Mehr Informationen dazu können im [Code Setup](#) gefunden werden.

6.4 Definition of Done

Damit die Use-Cases als abgeschlossen gelten, müssen folgende Kriterien erfüllt sein:

- Die im Use Case beschriebenen Funktionen wurden vollständig umgesetzt und verhalten sich wie spezifiziert.
- Für den jeweiligen Anwendungsfall wurden geeignete Tests implementiert und erfolgreich ausgeführt.
- Die Testergebnisse sind vollständig dokumentiert
- Die [Code Guidelines](#) wurden eingehalten.

7 Implementation

8 Fazit

Kapitel II

Projekt Dokumentation

9 Projekt Plan

9.1 Zusammenarbeit

Als Projektmanagement-Methode wird Kanban verwendet welches es erlaubt das Projekt agil durchzuführen. Aufgrund der Teamgrösse von zwei Personen wurde sich gegen Scrum entschieden. Der zusätzliche organisatorische Mehraufwand durch Daily Meetings, Sprint Reviews und weitere Scrum Meetings würde keinen grossen Nutzen bringen. Dank der geringen Teamgrösse ist es kein Problem den Überblick über das Projekt zu behalten, sodass eine schlanke Methode wie Kanban eine passende Wahl ist.

Die Zusammenarbeit und Aufgabenaufteilung wird mittels Jira organisiert.

9.2 Meetings

Jede Woche findet ein Meeting unter den Teammitgliedern und ein Meeting mit dem Referenten statt.

Im Team-Meeting wird besprochen, welche Aufgaben fertiggestellt wurden und diese gegebenenfalls zusammen anschaut. Zudem werden die Aufgaben für kommende Woche verteilt.

Beim Meeting mit dem Referenten wird der aktuelle Stand besprochen. Ausserdem kann dieses genutzt werden, um allfällige Fragen zu stellen.

9.3 Minimum Viable Product (MVP)

Das Minimum Viable Product (MVP) beinhaltet die Kernfunktionen des Projektes. Diese sind folgendermassen definiert:

- **Erweiterung Discord-Bot:** Ziel ist es, den bestehenden Discord-Bot basierend auf dem während des Einführungsprozesses für den Verein EGSwiss erhaltenen Feedback gezielt weiterzuentwickeln. Dabei sollen Funktionen neu erstellt, ergänzt und verbessert werden.
- **Erweiterung Gamification:** Ziel ist es, das bestehende Gamification-System auszubauen und zu untersuchen, welche Erweiterungen die Motivation und das Engagement im Verein steigern können.
- **Plattform-Unabhängigkeit:** Ein Admin-Dashboard soll erstellt werden, welches dem Vorstand Anpassungen an bereits erfassten Daten ermöglicht, ohne direkten Zugriff auf die Datenbank. Das Admin-Dashboard soll unabhängig von Discord nutzbar sein und somit die Applikation plattformunabhängig machen.

9.4 Long-Term Plan

Der Long-Term Plan wurde in 5 verschiedenen Phasen aufgeteilt. Diese werden in der folgenden Tabelle genauer beschrieben. Optionale Aufgaben stehen in Klammern.

Phase	Woche	Datum	Ziele	Meilenstein
Inception	1	15.09 - 21.09.	Einrichtung, MVP definieren	
	2	22.09. - 28.09.	Projektplan, Risikomanagement, Long-Term-Plan	M1
Elaboration	3	29.09. - 05.10.	Requirements, Einleitung, Stand der Technik	
	4	06.10. - 12.10.	Interviews andere Vereine, Sicherheitskonzept	
	5	13.10. - 19.10.	Gamificationkonzept, Architektur, Wireframes	
	6	20.10. - 26.10.	Testkonzept, Prototyp Admin-Dashboard (Proof of Concept)	M2
Construction	7	27.10. - 02.11.	Refactoring, 2.1, 2.3, 7.1	
	8	03.11. - 09.11.	1.1, 1.2, 3.1, 3.2, 3.3	
	9	10.11. - 16.11.	2.2, 4.1, 6.1, 7.2	
	10	17.11. - 23.11.	4.2, 4.3, 4.4, (5.1), (5.2), 6.2, (6.3)	M3
	11	24.11. - 30.11.	7.3, (7.4), 8.1, 8.2, (8.3)	
	12	01.12. - 07.12.	Feedback aus Usability Test umsetzen und Implementation beenden	M4
Transition	13	08.12. - 14.12.	Dokumentation, Abschluss	
	14	15.12. - 19.12.	Fertigstellung, Abgabe Abstract und Dokumentation	M5
Presentation	15	20.12. - 28.12.	Präsentation und A0-Poster	
	16	29.12.2025 - 04.01.2026		
	17	05.01. - 09.01.		M6

Tab. 47: Long-Term Plan

Die einzelnen Komponenten und Funktionen sind wie folgt definiert:

1. Backend

- **1.1:** Feedback / Neue Features ergänzen
- **1.2:** Auto. Zahlungsbestätigung über Bank

2. Datenbank

- **2.1:** Schema anpassen
- **2.2:** DB-Login verbessern (2FA)
- **2.3:** DB-Transactions / Rollbacks

3. Discord-Bot

- **3.1:** Feedback / Neue Features ergänzen
- **3.2:** Darstellung Events verbessern
- **3.3:** Logs einsehen

4. Admin-Dashboard

- **4.1:** Login erstellen
- **4.2:** Darstellung / Bearbeitung DB-Daten (Mitglieder, Rollen, Events, Logs)
- **4.3:** Erstellung des Punktesystems (Punkte, Belohnungen, Rangliste)
- **4.4:** Darstellung / Bearbeitung für Mitglieder
- **4.5:** Logs einsehen

5. Neue Plattform (optional)

- **5.1:** Wahl der Plattform
- **5.2:** Implementierung gleicher Features wie Discord-Bot

6. Unit-Tests

- 6.1: Discord-Bot
- 6.2: Dashboard
- 6.3: Neue Plattform

7. Pipelines CI/CD

- 7.1: Dokumentation
- 7.2: Discord-Bot
- 7.3: Dashboard
- 7.4: Neue Plattform

8. Usability Tests

- 8.1: Discord-Bot
- 8.2: Dashboard
- 8.3: Neue Plattform

9.4.1 Inception (Initiierung)

In der Inception-Phase werden die Anforderungen für das Projekt und der Projektplan erstellt, sowie die Risiken identifiziert. In dieser Phase wird zudem die generelle Herangehensweise und die Machbarkeit des Projektes bestimmt.

9.4.2 Elaboration (Ausarbeitung)

In der Elaboration-Phase werden die Anforderungen des Projektes im Detail analysiert. Hier wird das Gamification- und Sicherheitskonzept ausgearbeitet. Die Architektur wird mittels des C4-Modells definiert und ein erster Softwareprototyp wird erstellt. Dieser wird verwendet, um erste Systeme zu testen, auf denen dann später aufgebaut wird.

9.4.3 Construction (Konstruktion)

Die Construction-Phase wird dazu verwendet das Projekt technisch umzusetzen. In dieser Phase werden die geplanten Features umgesetzt und systematisch getestet. Usability-Tests werden durchgeführt auf dessen Basis Anpassungen vorgenommen werden. Die umgesetzten Features und Tests werden dokumentiert.

9.4.4 Transition (Übergang)

In der Transition-Phase wird das Projekt abgeschlossen. Die Dokumentation wird fertiggestellt und der Abstract wird geschrieben. Die Dokumentation wird in dieser Phase abgegeben.

9.4.5 Presentation (Präsentation)

Da das Projekt neben einer Studienarbeit auch eine Bachelorarbeit ist, wird in der Presentation-Phase die Präsentation erstellt. In dieser Phase arbeitet Vanessa Alves, welche die Bachelorarbeit durchführt, an der Präsentation und am Poster alleine.

9.4.6 Meilensteine

Die folgenden Meilensteine dienen als Hilfe, um die definierten Ziele des Projektes zu erreichen. Sie gelten als erfüllt, sobald die einzelnen Komponenten durch das Team in [Jira](#) als erledigt gekennzeichnet und vom Referenten eingesehen wurden.

M1: Projekt Initiierung

- Der Projektplan ist definiert.
- Die Risiken sind definiert.
- Das MVP ist definiert.
- Der Long-Term-Plan ist definiert.

M2: Prototyp

- Die Requirements sind definiert.
- Die Architektur ist definiert.
- Mockups für das Admin-Dashboard sind erstellt.
- Das Gamificationkonzept, Datenschutzkonzept und Testkonzept ist erstellt.
- Der erste Prototyp für das Admin-Dashboard ist erstellt.

M3: Erreichung des MVP

- Die Discord-Bot Erweiterungen sind umgesetzt.
- Das Admin-Dashboard ist umgesetzt.
- Das MVP wird erreicht.

M4: Fertigstellung der Implementation

- Alle geplanten, nicht-optionalen Features sind umgesetzt.
- Alle Funktionen sind erfolgreich getestet.
- Usability-Tests sind durchgeführt und Feedback ist umgesetzt.
- Die Testabdeckung entspricht dem definiertem Wert.

M5: Abgabe

- Die umfassende Dokumentation ist fertiggestellt.
- Das Abstract ist fertiggestellt.

M6: Präsentation

- Die Präsentation ist fertiggestellt.
- Das Poster ist fertiggestellt.

9.5 Risikomanagement

Die folgenden Tabellen zeigen die Risiken des Projektes auf und die Strategien die verwendet werden, um diese zu vermindern.

Wahrscheinlichkeit / Schweregrad	1 - Sehr Unwahrscheinlich	2 - Unwahrscheinlich	3 - Gelegentlich	4 - Wahrscheinlich	5 - Oft
4 - Katastrophal	R2				
3 - Kritisch		R3, R7	R9, R11		
2 - Signifikant			R4, R8, R13	R1, R12	
1 - Gering		R5		R6, R14	

Tab. 48: Risiko Matrix

ID	R1
Risiko	Lernkurve
Kommentar	Der Einsatz neuer Technologien (z. B. Frameworks, CI/CD, Web-Technologien) führt zu einer erhöhten Einarbeitungszeit.
Prävention	Frühzeitige Prototypen reduzieren die Einarbeitungszeit. Wissen wird im Team dokumentiert und geteilt.
Korrektur	Komplexe Features werden in kleinere Arbeitspakete unterteilt. Bei Problemen erfolgt gegenseitige Unterstützung im Team.

Tab. 49: Beschreibung Risiko 1

ID	R2
Risiko	Probleme bei Discord oder der neuen Plattformen
Kommentar	Discord oder alternative Plattformen (z. B. MS Teams, Matrix) können temporär nicht verfügbar sein.
Prävention	Evaluierung alternativer Plattformen und Absicherung kritischer Funktionen über die Web-Applikation.
Korrektur	Risiko muss weitgehend akzeptiert werden. Bei längeren Ausfällen wird die Nutzung der Web-Applikation verstärkt.

Tab. 50: Beschreibung Risiko 2

ID	R3
Risiko	Ressourcen limitiert
Kommentar	Ein Teammitglied fällt temporär oder dauerhaft aus.
Prävention	Alle Arbeitsschritte und Entscheidungen werden systematisch dokumentiert, um Wissenstransfer sicherzustellen.
Korrektur	Aufgaben werden durch verbleibende Teammitglieder übernommen. Fokus liegt auf priorisierten Features, um Projektziele trotz reduzierter Kapazität zu erreichen.

Tab. 51: Beschreibung Risiko 3

ID	R4
Risiko	Scope Creep
Kommentar	Anforderungen werden unklar definiert oder nachträglich erweitert, was den Projektumfang vergrössert.
Prävention	Detaillierte Anforderungsdefinition und Scope-Management im Rahmen der Meetings.
Korrektur	Nachträglich eingebrachte oder optionale Anforderungen mit niedriger Priorität werden in spätere Phasen verschoben oder gestrichen.

Tab. 52: Beschreibung Risiko 4

ID	R5
Risiko	Kompatibilitätsprobleme
Kommentar	Unterschiedliche Versionen von Tools, Bibliotheken oder Frameworks können zu Integrationsproblemen führen.
Prävention	Einheitliche Dokumentation der eingesetzten Versionen und Verwendung von Lockfiles (z. B. package-lock.json).
Korrektur	Versionskonflikte werden durch Abstimmung im Team gelöst, eine einheitliche Version wird verbindlich definiert.

Tab. 53: Beschreibung Risiko 5

ID	R6
Risiko	Zeiteinschätzung
Kommentar	Arbeitsaufwände werden zu optimistisch oder pessimistisch eingeschätzt.
Prävention	Iterative Abschätzung in den wöchentlichen Meetings sowie Nutzung von Erfahrungswerten und Vergleich mit vorherigen Projekten.
Korrektur	Optional geplante Features werden verschoben oder gestrichen, um Kernziele nicht zu gefährden.

Tab. 54: Beschreibung Risiko 6

ID	R7
Risiko	Änderungen in der Discord API oder andere Technologien
Kommentar	Fundamentale Änderungen in der Discord API oder anderen zentralen Technologien (z. B. Datenbank, Framework).
Prävention	Keine Verwendung von Funktionen, die als „deprecated“ markiert sind und Monitoring von Release Notes.
Korrektur	Anpassung des Codes und Refactoring bei API-Änderungen, auch wenn zusätzlicher Aufwand entsteht.

Tab. 55: Beschreibung Risiko 7

ID	R8
Risiko	Der bestehende Code ist stark an Discord gekoppelt, was die Entwicklung einer Web-App erschwert.
Kommentar	Der bisherige Code ist nicht komplett von Discord entkoppelt.
Prävention	Frühzeitiges Refactoring und Einführung einer modularen Architektur.
Korrektur	Falls Aufwand höher als geplant, Anpassung des Zeitplans und Reduktion optionaler Features.

Tab. 56: Beschreibung Risiko 8

ID	R9
Risiko	Dashboard ist nicht benutzerfreundlich
Kommentar	Dashboard erfüllen nicht die Erwartungen der Mitglieder/Vorstände hinsichtlich Usability.
Prävention	Entwicklung mit Usability-Tests und Einbindung von Stakeholdern.
Korrektur	Anpassung der Benutzeroberfläche und Interaktionskonzepte anhand von Feedback.

Tab. 57: Beschreibung Risiko 9

ID	R10
Risiko	Manipulation Gamification-System
Kommentar	Mitglieder könnten das Punktesystem missbrauchen (z. B. durch Mehrfachaktionen).
Prävention	Definition klarer Regeln, technische Prüfungen gegen Mehrfacheinträge, Monitoring verdächtiger Aktivitäten durch den Vorstand.
Korrektur	Rücksetzen oder Anpassung von Punkten bei Missbrauch und Anpassung der Regeln.

Tab. 58: Beschreibung Risiko 10

ID	R11
Risiko	Datenschutz
Kommentar	Potenzielle Risiken im Umgang mit personenbezogenen Daten.
Prävention	Erstellung eines Datenschutzkonzepts in Anlehnung an DSGVO und Schweizer Datenschutzgesetz sowie die Minimierung der erhobenen Daten.
Korrektur	Einholung von Einverständniserklärungen.

Tab. 59: Beschreibung Risiko 11

ID	R12
Risiko	Stakeholder-Anforderungen nicht berücksichtigt
Kommentar	Feedback von Mitgliedern oder Vorstand wird nicht ausreichend eingebunden.
Prävention	Priorisierung der Anforderungen, Usability-Tests mit realen Mitgliedern.
Korrektur	Anpassungen anhand von Feedback, Verschiebung von Features niedriger Priorität.

Tab. 60: Beschreibung Risiko 12

ID	R13
Risiko	Plattform-Integration komplexer als erwartet
Kommentar	Die geplante Entkopplung auf Dashboard oder anderen Plattformen verursacht höheren Aufwand oder technische Probleme.
Prävention	Proof-of-Concept frühzeitig erstellen, modulare Architektur.
Korrektur	Fokus auf Admin-Dashboard, weitere Plattformintegration in spätere Phase verschieben.

Tab. 61: Beschreibung Risiko 13

ID	R14
Risiko	Hosting oder CI/CD Ausfall
Kommentar	Server oder CI/CD-Pipeline fallen aus und blockieren Entwicklung oder Betrieb.
Prävention	Monitoring und automatisierte Backups, klare Verantwortlichkeiten im Team.
Korrektur	Deployment auf alternative Cloud-Umgebung oder manuelles Deployment als Notfalllösung.

Tab. 62: Beschreibung Risiko 14

9.5.1 Ausweichplan

Der Ausweichplan dient dazu systematisch auf Risiken zu reagieren, die im Vorfeld nicht bestimmt werden konnten. Der Ablauf ist wie folgt definiert:

1. Das Problem innerhalb einer Aufgabe wird identifiziert und mit dem Teammitglied besprochen.
2. Es wird versucht das Problem so schnell wie möglich zu lösen.
3. Kann das Problem nicht gelöst werden, wird es im nächsten Meeting mit dem Referenten besprochen.

Ziel dieses Ausweichplans ist es, bei auftretenden Problemen, die den Projektverlauf beeinträchtigen, eine klare und strukturierte Vorgehensweise zu gewährleisten. Dadurch kann effizient reagiert und ein reibungsloser Projektfortschritt sichergestellt werden.

10 Arbeitszeitnachweis

11 Tooling und Technologie-Entscheidungen

Dieses Kapitel beschreibt die im Rahmen der Bachelorarbeit eingesetzten Werkzeuge, Technologien und Entwicklungsprozesse. Es begründet die wesentlichen Entscheidungen hinsichtlich Projektorganisation, Dokumentation, Code-Qualität und Deployment. Grundlage bilden die Erfahrungen aus dem Vorprojekt, welche in dieser Arbeit systematisch weiterentwickelt werden.

11.1 Jira

Die Zusammenarbeit und Aufgabenverteilung wird mit Jira² verwaltet. Für jede Aufgabe wird im Kanban Board ein Issue erstellt mit den folgenden Merkmalen:

- **Epic:** Die Phase zu dem das Issue gehört.
- **Typ:** Der Typ des Issues (Meeting, Aufgabe).
- **Geschätzte Zeit:** Die Soll-Zeit des Issues.
- **Verwendete Zeit:** Die tatsächlich aufgewendete Zeit um, den Issue zu beenden.
- **Zugewiesene Person:** Die Person, die das Issue durchführt.

Die Issues werden alle zuerst im Backlog definiert. Im wöchentlichen Team-Meeting werden die Issues, welche in dieser Woche geplant sind in die „Todo“-Spalte geschoben. Wenn die Person, der das Issue zugeteilt wurde, mit der Arbeit beginnt, wird dieser in die „In Progress“-Spalte geschoben. Wurde das Issue umgesetzt, kommt er in die „Review“-Spalte. Alle Issues in der Review Spalte werden im wöchentlichen Meeting besprochen und bei einer erfolgreichen Umsetzung in die „Done“-Spalte verschoben. Falls es noch Verbesserungspotenzial gibt, kommt das Issue zurück in die „Todo“-Spalte.

11.2 Teams

Für die interne Kommunikation, spontane Absprachen und die Planung von ausserordentlichen Meetings wird Microsoft Teams³ verwendet. Es dient als zentraler Kommunikationskanal und Speicherort für geteilte Dokumente. Damit wird ein kontinuierlicher Informationsfluss sichergestellt, was insbesondere bei verteilter Arbeit im Rahmen der Bachelorarbeit entscheidend ist.

11.3 GitHub

Das Vorprojekt wurde zu Beginn der Arbeit von GitLab⁴ auf GitHub⁵ migriert, um die Entwicklungs- und Veröffentlichungsprozesse langfristig unabhängig von der OST-internen Infrastruktur zu gestalten. Die zuvor auf der OST-GitLab gehostete Umgebung war nur innerhalb der Hochschule zugänglich, wodurch externe Zusammenarbeit und eine spätere Öffnung des Projekts erschwert wurden. GitHub bietet im Gegensatz dazu eine öffentlich zugängliche, weit verbreitete Plattform, die sich in der Open-Source-Community etabliert hat. Durch die Nutzung von GitHub Organisations und integrierten CI/CD-Workflows (GitHub Actions) kann die Projektstruktur klar gegliedert und zukünftige Erweiterungen effizient verwaltet werden. Neben diesen Vorteilen diente der Wechsel ausserdem dem Erkenntnisgewinn für das Team.

Für die Projektstruktur wurde eine eigene Organisation namens „ClansCore“⁶ erstellt. Innerhalb dieser Organisation befinden sich die nachfolgenden Repositories:

Repository	Beschreibung
clanscore ⁷	Der Quellcode der Applikation.
clanscore-dok ⁸	Die Projek-Dokumentation.
clanscore-dok-pub ⁹	Die automatisch generierten PDF-Versionen der Dokumentation und Sitzungs-Protokolle, welche über GitHub Pages veröffentlicht werden (siehe <u>Abschnitt 11.4.2</u>).

²<https://www.atlassian.com/software/jira>

³<https://www.microsoft.com/de-ch/microsoft-teams/log-in?market=ch>

⁴<https://gitlab.ost.ch/>

⁵<https://github.com/>

⁶<https://github.com/ClansCore>

⁷<https://github.com/ClansCore/clanscore>

⁸<https://github.com/ClansCore/clanscore-doc>

Tab. 63: Übersicht der Repositories innerhalb der GitHub-Organisation ClansCore

Während der Bearbeitung dieser Arbeit sind die Haupt-Repositories privat gestellt, um den Entwicklungsprozess geschützt zu halten. Die Organisation selbst ist jedoch öffentlich, wodurch Transparenz, Auffindbarkeit und spätere Zusammenarbeit gewährleistet bleiben.

11.3.1 GitHub Guidelines

1. Auf dem Dokumentations-Repository hat jedes Teammitglied seinen eigenen Branch im Format „dev-Initialen“ (z.B. dev-jt)
2. Auf den Code-Repositories (Discord-Bot, Admin-Dashboard) soll für jedes Feature oder Fehler ein jeweiliger Branch im Format „feature-FeatureName“ oder „bug-BugName“ erstellt werden.
3. Das Deployment der Repositories wird je auf einem eigenen Branch namens „deploy“ umgesetzt. Die Sammlung der Commits werden beim mergen des Main-Banches mithilfe der Funktion „Squash“ zusammengefasst, um die Anzahl ähnlicher Commit-Nachrichten zu reduzieren.
4. Bevor Änderungen beim Code in den Develop-Branch gemerged werden, müssen diese von einem anderen Teammitglied überprüft werden.
5. Schreibe Commit-Nachrichten auf Englisch.
6. Beim wöchentlichen Meeting werden die Änderungen vom Develop-Branch in den Main-Branch gemerged.

11.4 Dokumentation

Die Projektdokumentation wird vollständig mit Typst¹⁰ erstellt. Die Entscheidung für Typst erfolgte aufgrund der einfacheren Syntax im Vergleich zu Latex¹¹, der nativen Git-Integration und der bereits im Vorprojekt begonnenen Typst-Vorlage.

11.4.1 Dokumentation Guidelines

1. Dokumentiere auf Deutsch.
2. Dokumentiere mit Typst.
3. Verwende die während diesem Projekt entwickelte Vorlage.
4. Unterteile den Text in eine Typst-Datei pro grösseres Kapitel.
5. Benenne die einzelnen Typst Dateien in Englisch und verwende hierfür kebab-case.
6. Verwende aussagekräftige Titel, welche den Text gut unterteilen.
7. Versehe alle nötigen Titel mit einer kurzen und klaren Referenz-Variable im snake_case.
8. Verwende eine saubere Formatierung, die dabei, hilft den Text gut lesen zu können.
9. Kennzeichne Referenzen aus externen Quellen als solche.
10. Schreibe für jede Abbildung und Tabelle eine Beschriftung.
11. Schreibe Referenz-Namen der Beschriftungen für Abbildungen und Tabellen im kebab-case.

11.4.2 Dokumentation Deployment

Im Rahmen dieser Arbeit wurde eine automatisierte Dokumentations-Pipeline implementiert, welche die kontinuierliche Erstellung und Veröffentlichung der Projektdokumentation gewährleistet. Ziel war es, den zuvor im Vorprojekt auf GitLab basierenden CI/CD-Prozess auf die GitHub-Infrastruktur zu migrieren und an die Anforderungen des neuen Projektsetups anzupassen (siehe Abschnitt 11.3).

Die Pipeline dient der automatischen Kompilierung und Veröffentlichung der Typst-Dokumentation, bestehend aus dem Hauptbericht und den Sitzungsprotokollen. Dabei werden sämtliche Schritte, vom Erstellen der PDF-Dateien bis zur Bereitstellung auf GitHub Pages, automatisiert ausgeführt. Da GitHub Pages für private Repositories in der kostenfreien Version nicht verfügbar ist, wurde eine Split-Repository-Strategie umgesetzt. Das private Repository (clanscore-doc) enthält den vollständigen Typst-Quellcode, während ein separates öffentliches Ziel-Repository (clanscore-doc-pub) ausschliesslich die generierten Ausgabedateien beherbergt.

⁹<https://github.com/ClansCore/clanscore-doc-pub>

¹⁰<https://typst.app/>

¹¹<https://www.latex-project.org/>

Die Pipeline wurde mittels GitHub Actions realisiert und befindet sich im privaten Repository. Der Workflow wird bei jedem Push auf den Main-Branch ausgelöst. Für die Authentifizierung und das Deployment wurde eine eigene GitHub App innerhalb der Organisation ClansCore erstellt. Diese App besitzt ausschliesslich die für das Deployment benötigten Berechtigungen auf das Ziel-Repository. Das App-Zugriffstoken wird zur Laufzeit über eine Action generiert und anschliessend verwendet, um den Inhalt des public/-Ordners in den Main-Branch des öffentlichen Repositories zu pushen.

Die GitHub Pages des Ziel-Repositories wurden so konfiguriert, dass der Main-Branch automatisch als statische Website veröffentlicht wird. Damit wird bei jeder Änderung im privaten Repository die Dokumentation neu erzeugt und unmittelbar online verfügbar gemacht.

Dieser Ansatz stellt eine klare und sichere CI/CD-Lösung dar. Der Quellcode und interne Inhalte bleiben privat, während die erzeugten Enddokumente öffentlich zugänglich sind. Durch die Verwendung einer GitHub App wird zudem auf persönliche Zugriffstokens verzichtet, was eine nachhaltige und wartungsarme Authentifizierung gewährleistet.

11.5 Code und Entwicklung

Der Code für dieses Projekt wird in der hier beschriebenen Form erstellt.

11.5.1 Code Guidelines

1. Schreibe TypeScript Variablen- und Funktionsnamen im camelCase.
2. Schreibe TypeScript Konstanten in Grossbuchstaben, wobei einzelne Wörter mit Unterstrichen aufgeteilt werden.
3. Schreibe HTML-Element-Bezeichner im kebab-case.
4. Gib Arrays einen Namen im Plural.
5. Schreibe kurze Funktionen mit wenigen Argumenten. Eine Funktion soll nur einen Zweck erfüllen.
6. Verwende let oder const anstelle von var.
7. Schreibe Vergleiche mit === oder !==.
8. Verwende bei Variablen-Zuweisungen Leerzeichen zwischen dem Gleichheitszeichen.
9. Schreibe bei Funktionsheadern die geschweifte Klammer auf die gleiche Zeile.
10. Schreibe Kommentare nur wo es nötig ist, der Code sollte selbsterklärend geschrieben werden.
11. Verwende für gleiche Konzepte die gleichen Wörter.
12. Überprüfe Änderungen vor dem pushen mit Prettier und ESLint.

11.5.2 Setup

Um den Coding-Style einzuhalten werden die Standardregeln von Prettier und ESLint für TypeScript verwendet. Zudem sind bei den Coding Projekten bestimmte Prozesse automatisiert, um die Entwicklung zu erleichtern.

11.5.3 Hosting

Um [ClansCore](#) dauerhaft online zu halten und beispielsweise auf Discord-Ereignisse zu reagieren oder zuverlässigen Zugriff auf das Admin-Dashboard zu gewährleisten, ist ein geeignetes Hosting erforderlich. Während dem [Vorprojekt](#) wurde der Discord-Bot auf einem Linux-Server des Vereins [EGSwiss](#) gehostet, durch manuelle Deployments über den Vereinsinternen Serveradmin. Da dieser Serveradmin zum Zeitpunkt der Arbeit anderweitig beschäftigt war sowie die Teammitglieder keinen direkten Zugriff auf den Server hatten, fiel die Entscheidung auf einen temporären virtuellen Server der Fachhochschule OST. Dieser läuft in einer stabilen Umgebung, ist kostenfrei und garantiert dem Team vollen Zugriff für die Einführung eines automatischen Deployments (siehe [Abschnitt 11.5.4](#)).

11.5.4 Code Deployment

Im [Vorprojekt](#) wurde das Deployment mit Docker manuell durch eine Drittperson auf einem dedizierten Linux-Server durchgeführt. Da ein manuelles Deployment fehleranfällig, langsam und ineffizient ist, wird ein automatisiertes Deployment umzusetzen ([AutoMQ-Contributors, 2025](#)).

Mit GitHub Actions kann das Deployment automatisiert und sichergestellt werden, dass bei einem Push auf den Main-Branch die Änderungen in die Produktivumgebung übernommen werden. Docker wird für das Containermanagement verwendet,

da es eine zuverlässige und plattformunabhängige Bereitstellung der Anwendung ermöglicht ([Docker-Contributors, 2025](#)) und das Team bereits Erfahrung mit Docker hat.

11.6 Datenbank

Im [Vorprojekt](#) wurde eine MongoDB-Datenbank verwendet. MongoDB ist eine dokumentenorientierte NoSQL-Datenbank, die Daten in JSON-ähnlichen Dokumenten speichert ([MongoDB-Contributors, 2025b](#)). Eine zentrale Funktion von MongoDB ist die horizontale Skalierbarkeit, was insbesondere bei einer steigenden Nutzung des Bots einen entscheidenden Vorteil darstellt. Die Erweiterungen, welche für ClansCore umgesetzt wurden, erforderten keine fundamentalen Änderungen an der Datenbank, sodass kein Anlass bestand, eine alternative Datenbanklösung in Erwägung zu ziehen.

11.7 Programmiersprachen und Frameworks

[ClansCore](#) baut auf einem bereits im [Vorprojekt](#) entwickelten Discord-Bot auf. Dieser wurde in TypeScript programmiert und verwendet die Bibliothek [Discord.js](#), welche als die am weitesten verbreitete und aktiv gepflegte Schnittstelle zur Discord API gilt ([Discord.js-Contributors, 2025a](#)). Die Bibliothek abstrahiert die Kommunikation über [REST-](#) und [WebSocket-Schnittstellen](#) und ermöglicht dadurch eine effiziente und stabile Implementierung der Bot-Funktionalitäten.

[Discord.js](#) wird fortlaufend weiterentwickelt und erhält regelmässig sicherheits- und funktionsbezogene Updates ([Discord.js-Contributors, 2025b](#)). Durch die positiven Erfahrungen aus dem Vorprojekt, den aktiven Community-Support und die hohe Kompatibilität mit TypeScript, wird [Discord.js](#) weiterhin als technologische Basis verwendet. Diese Entscheidung stellt sicher, dass bestehende Codebestandteile übernommen und nachhaltig weiterentwickelt werden können, ohne eine kostspielige Migration auf eine alternative Bibliothek durchführen zu müssen.

Für die Entwicklung des Admin-Dashboards standen verschiedene moderne Web-Technologien zur Auswahl. Da ein Framework den Entwicklungsprozess strukturiert, die Wiederverwendbarkeit erhöht sowie die Wartung erleichtert, wurde ein komponentenbasiertes Framework eingesetzt. Die Auswahl erfolgte anhand von Kriterien wie Verbreitung, Funktionsumfang, Zukunftssicherheit und vorhandene Teamerfahrung.

Web-Technologie	Beschreibung
React	Eine weit verbreitete JavaScript-Bibliothek für User Interfaces mit Millionen von Anwendern weltweit. React ist komponentenbasiert, kombiniert Logik und Markup in derselben Datei und profitiert von einer grossen Entwickler-Community. Die Flexibilität der Bibliothek ermöglicht unterschiedliche Architekturansätze, erfordert jedoch zusätzliche Tools für Routing, State-Management und Build-Prozesse. (React-Contributors, 2025)
Angular	Ein umfassendes Web-Framework, das eine vollständige Entwicklungsumgebung für skalierbare Web-Applikationen bietet. Angular wird von Google gepflegt, unterstützt TypeScript nativ und beinhaltet bereits Funktionen wie Routing, Dependency Injection und Formularverwaltung. Es ermöglicht eine klare Strukturierung und eine nachhaltige Code-Architektur. (Angular-Contributors, 2025)
Vue.js	Ein progressives JavaScript-Framework zur Entwicklung von User Interfaces. Vue kombiniert einfache Syntax mit hoher Flexibilität und eignet sich sowohl für kleine als auch komplexe Anwendungen. Es besitzt eine engagierte Community, wird jedoch primär von Einzelentwicklern und kleineren Teams gepflegt. (Vue.js-Contributors, 2025)

Tab. 64: Beschreibung der evaluierten Web-Technologien

Nach einer systematischen Bewertung wurde entschieden, das Admin-Dashboard mit Angular zu entwickeln. Diese Entscheidung beruht auf mehreren Faktoren:

- **TypeScript-Integration:** Angular ist vollständig in TypeScript implementiert, was eine einheitliche Codebasis zwischen Frontend und Backend ermöglicht. Dadurch können Typdefinitionen und Schnittstellen gemeinsam genutzt werden, was die Konsistenz und Fehlersicherheit erhöht.
- **Vollständige Framework-Struktur:** Im Gegensatz zu React, das primär eine UI-Bibliothek darstellt, bietet Angular ein integriertes Framework mit klaren Architekturvorgaben, standardisierten Projektstrukturen und eingebautem Tooling (z. B. Testing, Routing, Dependency Injection). Dies reduziert den Integrationsaufwand externer Bibliotheken und erhöht die Wartbarkeit.

- **Skalierbarkeit und Nachhaltigkeit:** Angular ist auf gross angelegte, modular aufgebaute Anwendungen ausgelegt und gewährleistet damit eine langfristig stabile Codebasis.
- **Teamkompetenz:** Beide Teammitglieder verfügten bereits über Erfahrung im Umgang mit Angular, was den Einarbeitungsaufwand erheblich reduzierte und einen produktiven Entwicklungsstart ermöglichte. Mit React und Vue.js liegen hingegen keine oder nur geringe praktische Erfahrungen vor.

Zusammenfassend wurde Angular aufgrund der klaren Struktur, nativen TypeScript-Unterstützung, hohen Skalierbarkeit und der bestehenden Teamexpertise als geeignetstes Framework für die Umsetzung des Admin-Dashboards bewertet. Diese Wahl stellt sicher, dass das Dashboard langfristig wartbar bleibt und sich konsistent in die bestehende ClansCore-Systemarchitektur integriert.

11.8 UI-Bibliothek

Bibliothekename	Beschreibung
Angular Material	Angular Material ist eine von der Angular-Entwicklergemeinschaft bereitgestellte Komponentenbibliothek, die auf den Richtlinien des Material Designs basiert. Sie bietet qualitativ hochwertige, internationalisierte und barrierefreie UI-Komponenten, die durch umfangreiche Tests auf Leistung und Zuverlässigkeit geprüft sind. (Material-Contributors, 2025)
PrimeNG	PrimeNG ist eine umfassende UI-Komponentenbibliothek für Angular, die eine Vielzahl an individuell anpassbaren und funktionsreichen Komponenten bereitstellt. Mit über 80 verfügbaren UI-Elementen unterstützt sie Entwicklerinnen und Entwickler bei der effizienten Umsetzung moderner Webanwendungen. (PrimeNg-Contributors, 2025)
NG-Zorro	NG-Zorro ist eine auf dem Ant Design basierende UI-Komponentenbibliothek für Angular, die speziell für den Unternehmenseinsatz entwickelt wurde. Sie stellt über 70 hochwertige, in TypeScript implementierte Komponenten bereit. (NG-ZORRO-Contributors, 2025)

Tab. 65: Bibliotheken für das User Interface

Im Unterschied zu PrimeNG und NG-Zorro fiel die Wahl auf Angular Material als UI-Komponentenbibliothek, da es eine besonders enge Integration in das Angular-Framework bietet und direkt vom Angular-Entwicklungsteam gepflegt wird. Dadurch ist eine hohe Kompatibilität mit aktuellen Angular-Versionen sowie eine langfristige Wartung und Stabilität gewährleistet.

Zudem überzeugt Angular Material durch seine klare Ausrichtung auf die Material-Design-Richtlinien, wodurch ein konsistentes und modernes Benutzererlebnis sichergestellt wird. Die Komponenten sind qualitativ hochwertig, barrierefrei und umfassend getestet, was insbesondere für produktive und langlebige Anwendungen von Vorteil ist.

11.9 KI-Tools

KI-Tools wurden eingesetzt, um die Effizienz zu steigern und die Qualität der Ergebnisse zu verbessern. Die Einsatzbereiche umfassen:

- **Technische Entscheidung:** Unterstützung bei der Bewertung von Frameworks, Bibliotheken und Architekturansätzen.
- **Programmierung:** Hilfe bei der Fehlersuche, Optimierung und Codegenerierung.
- **Dokumentation:** Unterstützung beim Strukturieren und Formulieren von Textabschnitten, insbesondere für technische Beschreibungen, Kapitelüberschriften und sprachliche Korrektheit.

Die KI wurde als Assistenzwerkzeug eingesetzt und diente nicht zur automatisierten Generierung vollständiger Inhalte. Alle durch die KI erzeugten Vorschläge wurden kritisch geprüft und bei Bedarf angepasst.

Folgende KI-Tools wurden verwendet:

- OpenAI ChatGPT¹²
- Google Gemini¹³
- GitHub Copilot¹⁴

¹²<https://chatgpt.com/>

¹³<https://gemini.google.com>

¹⁴<https://github.com/features/copilot>

12 Sitzungsprotokolle

Besprechung vom 19.09.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- Aufgabenstellung
 - Was kommt als nächstes
 - Abgabetermine
 - Feedback SA FS25
 - Sonstiges
-

Aufgabenstellung

Ein konkretes Projekt finden. Bei anderen onlinebasierten Vereinen Umfragen und Recherche durchführen um Konzept zu verbessern.

Eine Idee ist ein Admin-Dashboard zu machen um von Discord zu entkoppeln und keine Datenbankzugriffe mehr nötig sind. Eine Anbindung an Microsoft Teams oder die Open Source Plattform Matrix wäre denkbar.

Was kommt als nächstes

Problembeschreibung, Zeitplan und Meilensteine definieren.

Abgabetermine

Wegen der SA von Joel ist der Abgabetermin für die Arbeit am 19. Dezember und Vanessa hat noch für die Präsentation und Poster bis am 9. Januar Zeit.

Feedback SA FS25

Wird noch bereitgestellt.

Sonstiges

Änderungen zur Aufgabenstellung:

Gegenleser: Severin Dellsperger

Koreferent: Julia Czerniak-Wilmes

Besprechung vom 29.09.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- Besprechung aktueller Stand
 - ToDo
-

Besprechung aktueller Stand

- Meilensteine könnte man messbar machen (klar definieren wann diese erreicht sind)
- Im MVP sollte klar definiert sein welche Teile neu sind (Nur die Plattform-Unabhängigkeit in das MVP?)

ToDo

- Self-Determination-Theory: über dieses oder ähnliche Frameworks Research betreiben und in das Interview einbauen. (vorallem in Verbindung mit Competence, Autonomy, Relatedness)
- Problemstellung am Anfang sollte folgendes enthalten:
 - die zentralen technischen Problem finden und beschreiben. Wo könnte es Probleme geben und wieso ist es schwierig. (Kernprobleme, da unterschiedliche Lösungen finden und analysieren) (vlt Synchronisierung, Modularisierung?)
 - Alles Begründen. Alle Handlungsmöglichkeiten anschauen, analysieren und die Entscheidungen dokumentieren.
 - Das Produkt gut verkaufen. Was ist das endgültige Produkt. Gamification und Produkt gut zusammenpacken.

Besprechung vom 03.10.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- Besprechung aktueller Stand
-

Besprechung aktueller Stand

- Es wurde der aktuelle Stand geschildert.
- Verlängerung der Elaboration Phase und Verschiebung vom Gamification Konzept

Besprechung vom 10.10.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- GitHub Orga und CI/CD Doku
- Einleitung / Stand der Technik
- Anforderungen
- Interview andere Vereine
- Zwischenpräsentation

GitHub Orga und CI/CD Doku

Frieder wurde über die Organisation in GitHub informiert und die Möglichkeit Zugriff auf den Code und vor allem die online Version der aktuell im Main Branch deployte Dokumentation der Arbeit.

Organisation mit allen Repos: <https://github.com/ClansCore>

Einleitung / Stand der Technik

Wir haben zusammen die Einleitung und den Stand der Technik angeschaut und besprochen. Es ist nichts aufgefallen.

Anforderungen

Die Anforderungen (User Stories, FR, Use Cases und NFR) wurden angeschaut und besprochen.

Forlgende Anpassungen müssen noch gemacht werden:

- NFR3: Ist das ein Security Thema oder wie genau soll das gemessen werden? Evtl. Skript?
- NFR4: Welche UI-Regeln / Guidelines werden genau befolgt?
- NFR9: Messung muss genauer beschrieben werden

Interview andere Vereine

Die Interviews wurden durchgeführt und die unbearbeiteten Resultate vorgestellt. Diese werden diese Woche ausgewertet für das neue Gamification-Konzept.

Zwischenpräsentation

Vanessa wird voraussichtlich alleine die Präsentation Mitte Semester (Woche 7) durchführen. Joel darf freiwillig mitmachen oder zusehen.

Mögliche Daten für die Zwischenpräsentation, welche noch bestimmt wird.

- 28.10 - 15:00-17:00
- 29.10 - 13:00-15:00
- 30.10 - 13:00-15:00

Besprechung vom 17.10.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- Anforderungen
 - Gamification
 - Sicherheitskonzept
 - Wireframes
-

Anforderungen

Die Anforderungen (User Stories, FR, Use Cases und NFR) wurden angeschaut und besprochen.

- Es wurde entschieden, dass die Zahlungsbestätigung, Zahlungserinnerung und die Umsetzung der neuen Plattform eine tiefe Priorität bekommen und wahrscheinlich nicht umgesetzt werden. Folgende Anpassungen müssen noch gemacht werden:
- Die User Stories lösungsneutral halten (Kein Admin-Dashboard in der Beschreibung)
- FR12, FR13, FR14, FR15: Verschiebung in den Teil Implementation.
- FR16: Erklären wieso eine Implementation von einer weiteren Plattform möglich wäre, falls diese nicht umgesetzt wird. z.B durch gute Architektur/Schnittstellen Dokumentation

Gamification

Der aktuelle Stand wurde geschildert. Noch zu erledigen sind: Self-Determination Theory, Interview Auswertung, Analyse

Sicherheitskonzept

Der aktuelle Stand wurde angeschaut und besprochen. Es ist nichts aufgefallen..

- Severin darf kontaktiert werden und wegen dem Sicherheitskonzept gefragt werden.

Wireframes

Der aktuelle Stand wurde geschildert. Folgende Anpassungen müssen noch gemacht werden:

- Schildern wie man zum Design gekommen ist. (Beispiele, Forschung)

13 Persönliche Erfahrungsberichte

13.1 Bericht Joel Thoma

13.2 Bericht Vanessa Alves

Kapitel III

Glossar und Abkürzungsverzeichnis

ClansCore	ClansCore ist das Gesamt-System und der Produktname für die Erweiterung des Discord-Bots aus dem <u>Vorprojekt</u> . Es ist das End-Produkt dieser Arbeit und verbindet sämtliche Teil-Komponenten wie Backend, Datenbank, Discord-Bot, Admin-Dashboard und weitere Plattformen.
EGSwiss	Ein Schweizer Gaming Verein, wobei das Teammitglied Vanessa Alves Präsidentin ist und daher die nötige Expertise für die Vorarbeit und aktuelle Weiterentwicklung mitbringt sowie Erfahrungen aus dem Vereinsalltag einfließen lässt.
CI/CD	Continuous Integration (CI) und Continuous Deployment/Delivery (CD). Ein automatisierter Prozess, bei dem Codeänderungen kontinuierlich integriert, getestet und automatisch oder mit manueller Freigabe in die Produktionsumgebung publiziert werden.
REST-Schnittstelle	REST (Representational State Transfer) ist eine Architekturform für webbasierte Schnittstellen, bei der Daten über standardisierte HTTP-Methoden (z. B. GET, POST, PATCH, DELETE) ausgetauscht werden. REST-APIs sind zustandslos, d. h. jede Anfrage enthält alle notwendigen Informationen, um sie unabhängig von früheren Interaktionen zu verarbeiten. In <u>ClansCore</u> wird die Discord REST-API verwendet, um gezielte Aktionen wie das Senden von Nachrichten oder das Verwalten von Rollen durchzuführen.
WebSocket-Schnittstelle	WebSockets sind bidirektionales Kommunikationsprotokolle (RFC 6455), welches eine dauerhafte Verbindung zwischen Client und Server ermöglicht. Im Gegensatz zu REST, das auf einzelnen HTTP-Anfragen basiert, erlaubt WebSocket eine kontinuierliche Übertragung von Daten in Echtzeit. In <u>ClansCore</u> wird über das Discord-Gateway eine WebSocket-Verbindung aufgebaut, um Ereignisse (z. B. neue Nachrichten, Interaktionen oder Mitgliederänderungen) unmittelbar zu empfangen und darauf zu reagieren.
Role-Based Access Control (RBAC)	Role-Based Access Control (RBAC) ist ein Sicherheitskonzept, bei dem der Zugriff auf Ressourcen anhand von Rollen vergeben wird. Benutzer erhalten dabei bestimmte Rollen, die jeweils unterschiedliche Berechtigungen für Funktionen und Daten gewähren. So können Administratoren granular steuern, wer auf welche Teile eines Systems zugreifen darf.
Zwei- / Multi-Faktor-Authentifizierung (2FA/MFA)	Zwei- oder Multi-Faktor-Authentifizierung (2FA/MFA) ist ein Sicherheitsmechanismus, bei dem zur Anmeldung an einem System zwei verschiedene Authentifizierungsfaktoren abgefragt werden. Dies können z. B. ein Passwort und ein einmaliger Code (z. B. aus einer App oder per SMS) sein. 2FA beziehungsweise MFA erhöht die Sicherheit erheblich, da ein Angreifer beide Faktoren kennen oder besitzen müsste, um sich erfolgreich anzumelden.
Virtual Private Network (VPN)	Ein Virtual Private Network (VPN) ist eine Technologie, die eine verschlüsselte Verbindung über ein öffentliches oder unsicheres Netzwerk ermöglicht. Durch die Nutzung eines VPNs wird der Datenverkehr zwischen Client und Server gesichert, wodurch Vertraulichkeit und Integrität der übertragenen Daten gewährleistet werden.

Bastion Host		Ein <u>gehärteter Server</u> , der als sicherer Zugangspunkt zu einem internen Netzwerk dient. Er vermittelt administrative Zugriffe über einen kontrollierten, <u>MFA</u> -geschützten Kanal und verhindert direkten Zugriff auf interne Systeme.
Time-based Password (TOTP)	One-Time	Ein zeitbasiertes Einmalpasswort ist ein Verfahren zur Zwei-Faktor-Authentifizierung (2FA), bei dem temporäre, nur wenige Sekunden gültige Passwörter generiert werden. Diese werden auf einem Authenticator-Gerät oder in einer App erstellt. Die Synchronisation erfolgt über eine gemeinsame geheime Basis und die aktuelle Zeit, was eine zusätzliche Sicherheitsebene beim Login-Prozess bietet.
Phishing		Phishing bezeichnet den Versuch, über gefälschte E-Mails, Websites oder Nachrichten vertrauliche Informationen wie Passwörter, Kreditkartendaten oder Zugangsdaten zu erlangen. Angreifer nutzen hierbei Social-Engineering-Methoden, um das Vertrauen von Nutzern zu erschleichen. Ziel ist meist der unbefugte Zugriff auf Konten oder Systeme. Schulungen und Sensibilisierung helfen, Phishing-Angriffe zu erkennen und zu vermeiden.
CLOUD Act		Der Clarifying Lawful Overseas Use of Data Act (CLOUD Act) ist ein US-amerikanisches Gesetz von 2018, das US-Behörden den Zugriff auf Daten ermöglicht, die von US-Unternehmen gespeichert werden, auch wenn sich diese Daten physisch ausserhalb der USA befinden. Für in der Schweiz gehostete Systeme bedeutet dies, dass bei Nutzung von US-Cloud-Anbietern theoretisch ein ausländischer Datenzugriff möglich ist. Daher wird häufig auf Hosting in der Schweiz gesetzt, um die nationale Datenhoheit zu gewährleisten.
Systemhärtung (Hardening)	(Har-	Massnahmen zur Reduktion der Angriffsfläche eines Systems durch Entfernen unnötiger Dienste, Einschränkung von Benutzerrechten und konsequente Sicherheitskonfiguration. Ziel ist es, die Widerstandsfähigkeit gegen Angriffe zu erhöhen und unautorisierte Zugriffe zu verhindern.

Kapitel IV

Literaturverzeichnis

- Alves, V., & Schnider, M. (2025). *Discord-Bot für Vereine* [Studien- und Bachelorarbeit].
- Angular-Contributors. (2025,). *What is Angular?*. <https://angular.dev/overview>
- Arcane-Contributors. (2025,). *Arcane Discord Bot*. <https://arcane.bot/>
- AutoMQ-Contributors. (2025,). *Common Issues with Manual Deployment*. <https://www.automq.com/blog/fully-automated-deployment-vs-manual-deployment-comparison#common-issues-with-manual-deployment>
- Chou, Y.-k. (2015). *Actionable Gamification: Beyond Points, Badges and Leaderboards* (, Hrsg.). CreateSpace Independent Publishing Platform.
- CISA. (2022). *Implementing Multi-Factor Authentication (MFA) for Remote Access and VPNs* [Technical report]. https://www.cisa.gov/sites/default/files/publications/CISA_MFA_Guidelines_Remote_Access_2022.pdf
- ClubDesk-Contributors. (2025,). *Vereinssoftware für die einfache Vereinsverwaltung*. <https://www.clubdesk.de/de/startseite>
- Discord.js-Contributors. (2025b,). *Commits*. <https://github.com/discordjs/discord.js/commits/main/>
- Discord.js-Contributors. (2025a,). *The most popular way to build Discord bots*. <https://discord.js.org/>
- Docker-Contributors. (2025,). *Developers bring their ideas to life with Docker*. <https://www.docker.com/why-docker/>
- Europäische-Union. (2016,). *VERORDNUNG (EU) 2016/679 DES EUROPÄISCHEN PARLAMENTS UND DES RATES*. <https://eur-lex.europa.eu/legal-content/DE/TXT/?uri=CELEX:32016R0679>
- Jakob, N. (1993,). *Response Times: The 3 Important Limits*. Nielsen Norman Group. <https://www.nngroup.com/articles/response-times-3-important-limits/>
- madewithangular-Contributors. (2025,). *A showcase of web apps made with Angular*. <https://www.madewithangular.com/sites>
- Martin, F. (2003,). *Optimistic Offline Lock*. <https://martinfowler.com/eaCatalog/optimisticOfflineLock.html>
- Material-Contributors, A. (2025,). *Angular Material - Material Design components for Angular*. <https://material.angular.dev/>
- MEE6-Contributors. (2025,). *Statistics Channels Plugin*. <https://wiki.mee6.xyz/en/plugins/statistics-channels>
- Microsoft-Contributors. (2025b,). *Planning for mandatory multifactor authentication for Azure and other admin portals*. <https://learn.microsoft.com/en-us/entra/identity/authentication/concept-mandatory-multifactor-authentication?tabs=dotnet>
- Microsoft-Contributors. (2025a,). *Security at your organization: Multifactor authentication statistics*. <https://learn.microsoft.com/en-us/partner-center/security/security-at-your-organization>
- molily. (2025,). *Running the unit and integration tests*. <https://testing-angular.com/angular-testing-principles/#running-the-unit-and-integration-tests>
- MongoDB-Contributors. (2025d,). *Authentication and Authorization with OIDC/OAuth 2.0*. <https://www.mongodb.com/docs/manual/core/oidc/security-oidc/>
- MongoDB-Contributors. (2025b,). *Introduction to MongoDB*. <https://www.mongodb.com/docs/manual/introduction/>
- MongoDB-Contributors. (2025f,). *Role-Based Access Control in Self-Managed Deployments*. <https://www.mongodb.com/docs/manual/core/authorization/>
- MongoDB-Contributors. (2025e,). *SCRAM*. <https://www.mongodb.com/docs/manual/core/security-scam/>
- MongoDB-Contributors. (2025a,). *Transactions*. <https://www.mongodb.com/docs/manual/core/transactions/>
- MongoDB-Contributors. (2025c,). *Use X.509 Certificates to Authenticate Clients on Self-Managed Deployments*. <https://www.mongodb.com/docs/manual/tutorial/configure-x509-client-authentication/>

- Murugiah, S., & Karen, S. (2017). *Application Container Security Guide (NIST SP 800-190)* [Technical report]. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-190.pdf>
- NG-ZORRO-Contributors. (2025,). *Angular UI Component Library based on Ant Design*. <https://github.com/NG-ZORRO/ng-zorro-antd>
- NIST. (2020). *Security and Privacy Controls for Information Systems and Organizations (SP 800-53 Rev. 5)* [Technical report]. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-53r5.pdf>
- OWASP-Cheat-Sheets-Series-Contributors. (2025,). *Session Management Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- OWASP-Contributors. (2021,). *OWASP Top 10*. <https://owasp.org/Top10/>
- Peters Dorian, R. R. M., Calvo Rafael A. (2018). Designing for Motivation, Engagement and Wellbeing in Digital Experience. *Frontiers in Psychology*, 9. <https://www.frontiersin.org/journals/psychology/articles/10.3389/fpsyg.2018.00797>
- PrimeNg-Contributors. (2025,). *The Next-Gen UI Suite for Angular*. <https://primeng.org/>
- React-Contributors. (2025,). *React*. <https://react.dev/>
- Schweizerische-Eidgenossenschaft. (2020,). *Bundesgesetz über den Datenschutz (Datenschutzgesetz, DSG)*. <https://www.fedlex.admin.ch/eli/cc/2022/491/de>
- U.S.-Department-of-Justice. (2019,). *The Purpose and Impact of the CLOUD Act - FAQs*. <https://www.justice.gov/criminal/media/999616/dl?inline>
- Vue.js-Contributors. (2025,). *What is Vue?*. <https://vuejs.org/guide/introduction>
- Weir, K. (2025,). *Self-determination theory: A quarter century of human motivation research*. <https://www.apa.org/research-practice/conduct-research/self-determination-theory>
- WildApricot-Community. (2015,). *Gamify Member's Site Participation*. <https://forums.wildapricot.com/forums/308932-wishlist/suggestions/10321731-gamify-member-s-site-participation>
- WildApricot-Contributors. (2025,). *Membership Management Software*. <https://www.wildapricot.com/>

Kapitel V

Abbildungsverzeichnis

Abb. 1	Die Bewertung des Vorprojektes im Octalysis-Modell	18
Abb. 2	Domain Model	27
Abb. 3	System Context Diagramm	28
Abb. 4	Container Diagramm	29
Abb. 5	Login	30
Abb. 6	Bewerbungsformular	30
Abb. 7	Ranglisten-Seite	31
Abb. 8	Aufgaben-Seite	31
Abb. 9	Mitglieder-Seite	31
Abb. 10	Punktedefinitions-Seite	32
Abb. 11	Jahresplanung-Seite	32
Abb. 12	Belohnung-Seite	32
Abb. 13	Punktebeispiel-Seite	33
Abb. 14	Statistiken-Seite	33

Kapitel VI

Tabellenverzeichnis

Tab. 1	Beschreibung der Prioritäts-Kategorien	4
Tab. 2	Beschreibung der Resultats-Kategorien	5
Tab. 3	Beschreibung Functional Requirement 1	6
Tab. 4	Beschreibung Functional Requirement 2	6
Tab. 5	Beschreibung Functional Requirement 3	6
Tab. 6	Beschreibung Functional Requirement 4	6
Tab. 7	Beschreibung Functional Requirement 5	6
Tab. 8	Beschreibung Functional Requirement 6	6
Tab. 9	Beschreibung Functional Requirement 7	7
Tab. 10	Beschreibung Functional Requirement 8	7
Tab. 11	Beschreibung Functional Requirement 9	7
Tab. 12	Beschreibung Functional Requirement 10	7
Tab. 13	Beschreibung Functional Requirement 11	7
Tab. 14	Beschreibung Functional Requirement 12	7
Tab. 15	Beschreibung Fully dressed Use Case 1	8
Tab. 16	Beschreibung Fully dressed Use Case 2	8
Tab. 17	Beschreibung Fully dressed Use Case 3	9
Tab. 18	Beschreibung Fully dressed Use Case 4	9
Tab. 19	Beschreibung Fully dressed Use Case 5	9
Tab. 20	Beschreibung Fully dressed Use Case 6	10
Tab. 21	Beschreibung Fully dressed Use Case 7	10
Tab. 22	Beschreibung Fully dressed Use Case 8	10
Tab. 23	Beschreibung Fully dressed Use Case 9	11
Tab. 24	Beschreibung Fully dressed Use Case 10	11
Tab. 25	Beschreibung Fully dressed Use Case 11	12

Tab. 26 Beschreibung Non-Functional Requirement 1	12
Tab. 27 Beschreibung Non-Functional Requirement 2	12
Tab. 28 Beschreibung Non-Functional Requirement 3	13
Tab. 29 Beschreibung Non-Functional Requirement 4	13
Tab. 30 Beschreibung Non-Functional Requirement 5	13
Tab. 31 Beschreibung Non-Functional Requirement 6	13
Tab. 32 Beschreibung Non-Functional Requirement 7	14
Tab. 33 Beschreibung Non-Functional Requirement 8	14
Tab. 34 Beschreibung Non-Functional Requirement 9	14
Tab. 35 Beschreibung Non-Functional Requirement 10	14
Tab. 36 Mapping der Interviewergebnisse zu den SDT-Grundbedürfnissen	16
Tab. 37 Die Bewertung des Vorprojektes im Octalysis-Modell	18
Tab. 38 Leitprinzipien für ClansCore	19
Tab. 39 Erweiterungen und Zielwirkung	20
Tab. 40 Beschreibung der vier grundlegenden Prinzipien	22
Tab. 41 Beschreibung der abgesicherten Kanäle in ClansCore	24
Tab. 42 Einschätzung der Risiken und deren Massnahmen	25
Tab. 43 Ablauf Unit-Tests	35
Tab. 44 Ablauf Integration-Tests	35
Tab. 45 Ablauf System-Tests	35
Tab. 46 Ablauf Usability-Test	35
Tab. 47 Long-Term Plan	40
Tab. 48 Risiko Matrix	42
Tab. 49 Beschreibung Risiko 1	42
Tab. 50 Beschreibung Risiko 2	43
Tab. 51 Beschreibung Risiko 3	43
Tab. 52 Beschreibung Risiko 4	43
Tab. 53 Beschreibung Risiko 5	43
Tab. 54 Beschreibung Risiko 6	43

Tab. 55 Beschreibung Risiko 7	44
Tab. 56 Beschreibung Risiko 8	44
Tab. 57 Beschreibung Risiko 9	44
Tab. 58 Beschreibung Risiko 10	44
Tab. 59 Beschreibung Risiko 11	44
Tab. 60 Beschreibung Risiko 12	44
Tab. 61 Beschreibung Risiko 13	45
Tab. 62 Beschreibung Risiko 14	45
Tab. 63 Übersicht der Repositories innerhalb der GitHub-Organisation ClansCore	47
Tab. 64 Beschreibung der evaluierten Web-Technologien	50
Tab. 65 Bilbiotheken für das User Interface	51
Tab. 66 Varianten zur Absicherung des DB-Zugangs	67

Kapitel VII

Code-Listings

Kapitel VIII

Anhang

14 Interviews zur Vereins-Motivation

Die Interviews wurden als halbstrukturierte, qualitative Gespräche mit vier anonymisierten Präsidenten Schweizer Gaming-Vereine geführt (durchschnittlich ungefähr 40 Minuten, leitfadengestützt). Die Auswahl erfolgte gezielt, um organisatorische und strategische Perspektiven abzubilden.

Der Leitfaden war entlang der drei Grundbedürfnisse der Self-Determination Theory (SDT: Autonomie, Kompetenz, soziale Eingebundenheit) strukturiert und wurde um praxisbezogene Blöcke (Kommunikation, digitale Tools, Fairness / Regeln, Kennzahlen) ergänzt. Die Auswertung erfolgte qualitativ-interpretativ in Form einer abgeleiteten Kodierung nach SDT, ergänzt um darauf aufbauende Subcodes (z. B. „Kommunikationsengpass“, „Anerkennung sichtbar machen“, „Saisonalität / Resets“, „Datensparsamkeit“).

Zur Sicherstellung eines klaren Aufbaus wurde der Leitfaden in sieben thematische Blöcke gegliedert:

1. **Allgemeine Fragen:** Kontext zu Rolle, Vereinsgrösse und Gründungsjahr zur Vergleichbarkeit der Antworten.
2. **Vereinsmotivation:** Aktuelle Beitrittsgründe, Bindungsfaktoren und Herausforderungen.
3. **Aktivitäten und Engagement:** Motivierende Angebote, Unterschiede zwischen neuen und erfahrenen Mitgliedern, Umgang mit Passivität.
4. **Digitale Tools und Gamification:** Nutzung digitaler Plattformen, Erfahrungen mit spielerischen Elementen, Chancen und Risiken.
5. **Zukunft und Anpassungen:** Wünsche nach Belohnungen und nachhaltigen Fördermassnahmen.
6. **Digitale Brücke:** Hypothetische Fragen zu einem digitalen Tool, Kennzahlen und gewünschten Features.
7. **Abschluss:** Offene Ergänzungen und Reflexionen.

Um unbeeinflusste Antworten zu gewährleisten, wurden digitale Lösungsansätze erst in den abschliessenden Blöcken thematisiert. Die vollständigen Leitfadenfragen sowie die anonymisierten Rohantworten und die Kodiermatrix sind als separate Dateien im Anhang abgelegt (vgl. „Gamification Interviews - Plan.pdf“ und „Gamification Interviews - Antworten.xlsx“).

Eine zusammenfassende Analyse der Ergebnisse erfolgt in [Abschnitt 3.2](#) des erweiterten Gamification-Konzeptes.

15 Technische Ergänzungen zum Sicherheitskonzept

Dieses Kapitel enthält technische Detailinformationen zum erweiterten Sicherheitskonzept ([Abschnitt 4](#)). Die hier dargestellten Inhalte dienen der Nachvollziehbarkeit der sicherheitsrelevanten Implementierungsaspekte und unterstützen die im Hauptteil beschriebenen konzeptionellen Massnahmen.

15.1 Vergleich der DB-MFA-Varianten

In der folgenden Tabelle sind die untersuchten MFA-Varianten für die in diesem Projekt verwendete MongoDB dargestellt:

Variante	Beschreibung	Stärken (+) und Schwächen (-)
Netzwerk-MFA (VPN / SSH-Bastion)	Zugriff auf die Datenbank ist nur über ein privates, nicht öffentlich erreichbares Netzwerk möglich. Der Zugang zu diesem Netzwerk wird über eine Bastion oder einen VPN-Gateway geregelt, der eine Multi-Faktor-Authentifizierung (z. B. TOTP oder Hardware-Token) verlangt. Damit wird der gesamte Zugriffspfad geschützt, ohne dass Backend oder Bot angepasst werden müssen. (Bezug: NFR6 , NFR9)	+ geringer Implementierungsaufwand, hohe Kompatibilität und transparente Integration in bestehende Infrastruktur. - VPN oder Bastion werden zu zentralen Kontrollpunkten, die gezieltes Monitoring und Backup-Strategien erfordern.
X.509-Clientzertifikate (mutual TLS)	Die Authentifizierung erfolgt über ein digitales Zertifikat als Besitzfaktor (MongoDB-Contributors, 2025c) (Bezug: NFR6). Diese Methode bietet einen hohen Schutz vor Phishing und erlaubt eine präzise Nachverfolgbarkeit einzelner Zugriffe.	+ stark gegen Phishing und gute Auditierbarkeit. - aufwändige Zertifikatsverwaltung, zusätzlicher Administrationsaufwand.
SSO (Single-Sign-On) / Identity-Provider- basierte MFA	Authentifizierung erfolgt über einen externen Identity Provider (z. B. Microsoft Entra ID ¹⁵) mittels OpenID Connect (nachfolgend OIDC genannt). MongoDB unterstützt seit Version 7.0 die direkte Integration solcher Anbieter, wodurch sich SSO und MFA zentral verwalten lassen (MongoDB-Contributors, 2025d) (Bezug: NFR6).	+ zentrale Benutzer- und Richtlinienverwaltung über Systeme hinweg. - erfordert zusätzliche Infrastruktur, Lizenzkosten und laufende Wartung.

Tab. 66: Varianten zur Absicherung des DB-Zugangs

Die Alternative über X.509-Zertifikate als besonders sicher eingestuft, ist aber aufgrund des erhöhten Verwaltungsaufwands und der nötigen Zertifikatsinfrastruktur für den Anwendungsfall eines Vereins-Tools nur bedingt sinnvoll.

Eine SSO- / IdP-basierte Lösung bietet theoretisch die beste Richtliniensteuerung und zentrale Benutzerverwaltung, würde aber zusätzliche Infrastruktur, Lizenzen und Wartungskosten verursachen. Da ClansCore bewusst als Open-Source-System für Vereine konzipiert ist, stehen Kosteneffizienz und einfache Wartbarkeit im Vordergrund. Die Einführung eines kommerziellen Identity-Providers würde diesen Grundsatz unterlaufen und die Einstiegshürde für kleinere Vereine erheblich erhöhen.

Aus diesen Gründen wird für ClansCore die Lösung Netzwerk-MFA über VPN beziehungsweise SSH-Bastion gewählt. Diese Variante bietet eine robuste Absicherung gegen unautorisierte Zugriffe, ist wirtschaftlich umsetzbar und unterstützt das Prinzip der klaren Trennung zwischen menschlichen und maschinellen Zugängen. Menschliche Admins authentifizieren sich über MFA auf der Netzwerkebene und systeminterne Dienste (z. B. Bot, Backend) kommunizieren ausschliesslich innerhalb des privaten Docker-Netzwerks über dedizierte Dienstkonten mit minimalen Rechten.

Die Wirksamkeit dieser Architektur wird durch externe Quellen untermauert. Laut Microsoft verhindern aktivierte MFA-Mechanismen ca. 99.2% bis 99.9% der üblichen Kontoübernahmen und werden deshalb in Zukunft potenziell zwingend eingeführt für VPN / Bastion ([Microsoft-Contributors, 2025a; 2025b](#)) . Ausserdem empfiehlt die Cybersecurity and Infrastructure Security Agency (CISA) den MFA-Einsatz, insbesondere für VPN- und Bastion-Zugänge ([CISA, 2022](#)) . Damit erfüllt die gewählte Lösung die anerkannten Best Practices moderner IT-Sicherheitsarchitekturen und bleibt zugleich ressourcenschonend, was den spezifischen Anforderungen einer Vereinssoftware gerecht wird.

Zusätzlich nutzt die Datenbank das standardisierte Authentifizierungsverfahren SCRAM-SHA-256 ([MongoDB-Contributors, 2025e](#)) in Kombination mit rollenbasierter Zugriffskontrolle ([MongoDB-Contributors, 2025f](#)) . Für zukünftige Ausbaustufen kann eine ergänzende Zertifikatsauthentifizierung für besonders privilegierte Konten in Betracht gezogen werden.

15.2 Dashboard-Sicherheit und Passwortmanagement

Nach erfolgreicher Anmeldung werden kurzlebige Zugriffstokens und rotierende Auffrischungstokens ausgestellt. Tokens werden als HttpOnly- und Secure-Cookies gespeichert und zustandsverändernde Anfragen erfordern ein CSRF-Token. Token-Lebenszyklen und Session-Invalidierung entsprechen den Empfehlungen des OWASP zur sicheren Sitzungsverwaltung ([OWASP-Cheat-Sheets-Series-Contributors, 2025](#)) .

¹⁵<https://www.microsoft.com/de-ch/security/business/identity-access/microsoft-entra-id>

Sämtliche Kommunikation zwischen Frontend und Backend erfolgt über TLS-verschlüsselte HTTPS-Verbindungen. Rollen- und Berechtigungsprüfungen erfolgen serverseitig über RBAC.

Vergessene Passwörter können über einen zeitlich begrenzten Reset-Link (Gültigkeit ≤ 30 Minuten) zurückgesetzt werden. Reset-Links werden ausschliesslich an die hinterlegte E-Mail-Adresse gesendet, ohne offenzulegen, ob ein Konto existiert. Die Passwort-Policy fordert mindestens 12 Zeichen, Kombination aus Gross- / Kleinbuchstaben, Ziffern und Sonderzeichen. Passwörter werden mit bcrypt (≥ 12 Rounds) oder einem gleichwertigen KDF sicher gehasht gespeichert. Fehlgeschlagene Anmeldeversuche führen zu progressiven Delays (Rate-Limiting). Erfolgreiche und fehlerhafte Änderungen werden im Audit-Log protokolliert.

Diese technischen Massnahmen dienen der Umsetzung der Anforderungen [NFR3](#), [NFR6](#) und [NFR9](#) und stellen sicher, dass Authentifizierungsvorgänge den Prinzipien von Privacy by Design und Defense in Depth entsprechen.

15.3 Nebenläufigkeitskontrolle und Transaktionen

Jeder Datensatz in der Datenbank enthält eine Versionskennung sowie einen Zeitstempel. Bei einem Update-Vorgang überprüft das Backend, ob die vom Client übermittelte Version noch mit der aktuellen Version in der Datenbank übereinstimmt. Wurde der Datensatz in der Zwischenzeit verändert, wird der Schreibvorgang abgewiesen und als Konflikt im Audit-Log vermerkt. Das Dashboard fordert daraufhin den aktuellen Datensatz an und kann die Änderungen manuell oder automatisch zusammenführen.

Für besonders sensible Vorgänge, wie Punktebuchungen, Rollenänderungen oder Event-Updates, werden zusätzlich atomare Transaktionen eingesetzt. Dadurch ist gewährleistet, dass zusammenhängende Operationen entweder vollständig oder gar nicht ausgeführt werden, was Inkonsistenzen in der Datenbank verhindert.

Alle Konflikte sowie deren Auflösung werden im Audit-Log dokumentiert.

15.4 Container- und Deployment-Details

Die Architektur besteht aus separaten Containern für Discord-Bot, Backend-Service, Admin-Dashboard, Reverse-Proxy und MongoDB. Jeder Container besitzt ein eigenes Dateisystem, eigene Prozessräume und dedizierte Netzwerk-Namespaces. Die Kommunikation erfolgt ausschliesslich über interne Docker-Netzwerke. Externe Zugriffe laufen ausschliesslich über den Reverse-Proxy mit TLS 1.3-Verschlüsselung. Die Datenbank ist nicht öffentlich erreichbar (siehe [Abschnitt 4.3.2](#)).

Container-Isolation basiert auf Linux-Namespaces, Cgroups und Capability-Drops, die laut NIST als Kernmechanismen moderner Container-Sicherheit gelten sowie die Angriffsfläche erheblich reduzieren. ([Murugiah & Karen, 2017](#))

Zugriff auf den Host erfolgt ausschliesslich über SSH-Key-Authentifizierung mit MFA über VPN / Bastion-Ebene ([CISA, 2022](#)). Das Deployment erfolgt automatisiert über GitHub Actions, wobei sämtliche Secrets (z. B. Tokens, Zugangsdaten) in verschlüsselten GitHub-Vaults verwaltet werden. Backups sind versioniert, verschlüsselt und nur für Admins zugänglich.